Computers & Industrial Engineering 210 (2025) 111547

Contents lists available at ScienceDirect

Computers & Industrial Engineering

journal homepage: www.elsevier.com/locate/caie

# A Q-learning based large neighborhood search algorithm for solving food delivery routing problem with uncertain demand and service time

Liyang Xiao [a], Zehao Du [a], Peng Sun [b,*], Shi Cheng [c,*]

[a] School of Management, Shanghai University, Shanghai, China
[b] CoME, Tianjin University, Tianjin, China
[c] School of Artificial Intelligence and Computer Science, Shaanxi Normal University, Xi'an, China

ARTICLE INFO

*Keywords:*
Food delivery
Robust optimization
Uncertainty
Q-learning
Large neighborhood search

ABSTRACT

With the widespread adoption of smartphones and advancements in internet technology, the food delivery industry has experienced rapid growth. However, fluctuations in customer demand and the uncertainty in rider service times pose significant challenges to delivery efficiency. This paper uses a robust optimization approach to address the food delivery route optimization problem. A robust optimization model is proposed that considers uncertain demand and service times to minimize platform operational costs through optimized order allocation and route planning. To solve this problem, a Q-learning based large neighborhood search algorithm (QL-LNS) is proposed, which learns effective operator selection strategies from historical optimization knowledge to improve solution quality and computational efficiency. Experimental results demonstrate that the proposed algorithm yields results consistent with CPLEX in small-scale instances and outperforms the adaptive large neighborhood search (ALNS) algorithm in large-scale instances. Additionally, the robust optimization model exhibits strong resilience to demand fluctuations and variations in service times. Although the robust approach may lead to a slight increase in costs, it better adapts to uncertainty changes, enhancing overall performance.

## 1. Introduction

The proliferation of smartphones and advancements in internet technology have significantly increased the convenience of food delivery, driving a substantial shift from traditional phone orders to digital platforms (Zhen et al., 2022). With the accelerated pace of modern life, food delivery has become a common consumption trend. Customers can now use their smartphones to place orders with a few simple actions and receive meals at designated locations within scheduled times. This convenience has significantly met daily dining needs and laid a solid foundation for the rapid growth of food delivery platforms. Companies such as Meituan (China), DoorDash (United States), Deliveroo (United Kingdom), and Uber Eats exemplify this growth. For instance, in 2023, Meituan's revenue reached 276.7 billion RMB, reflecting a 26% year-over-year increase. The rapid expansion of the food delivery industry highlights sustained consumer demand for convenient dining options.

Typically, the operational flow of online food delivery platforms proceeds as follows: customers select menu items and place orders through the platform's application, which receives a high volume of orders in real time. Platform-specific algorithms then match orders with riders, assign tasks, and plan delivery routes. Once the restaurant receives the order, they prepare the food, while the rider follows the designated route to pick up the order and deliver it to the customer within the specified time window. Fig. 1 illustrates a sample delivery route.

In practice, food delivery platforms often receive a large volume of orders within short time frames, necessitating the efficient allocation of orders to different riders and optimal route planning to balance relationships among restaurants, riders, and customers. Effective route optimization not only enhances operational efficiency and reduces costs but also improves the customer experience. In an increasingly competitive market, the challenge of how to improve delivery efficiency while meeting customers' demand for timely service has become a significant area for further research.

However, several challenges arise in the delivery process, with the most prominent being the impact of stochastic factors, such as fluctuations in customer demand and rider service times (Zhen et al., 2023). To simplify analysis, it is sometimes assumed that all data are deterministic and fully known, allowing for the construction of deterministic mathematical models. However, real-world conditions are typically more complex. Given the random nature of order demand,

* Corresponding authors.
*E-mail addresses:* sunpeng1317@tju.edu.cn (P. Sun), cheng@snnu.edu.cn (S. Cheng).

https://doi.org/10.1016/j.cie.2025.111547
Received 14 January 2025; Received in revised form 15 July 2025; Accepted 18 September 2025
Available online 24 September 2025
0360-8352/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

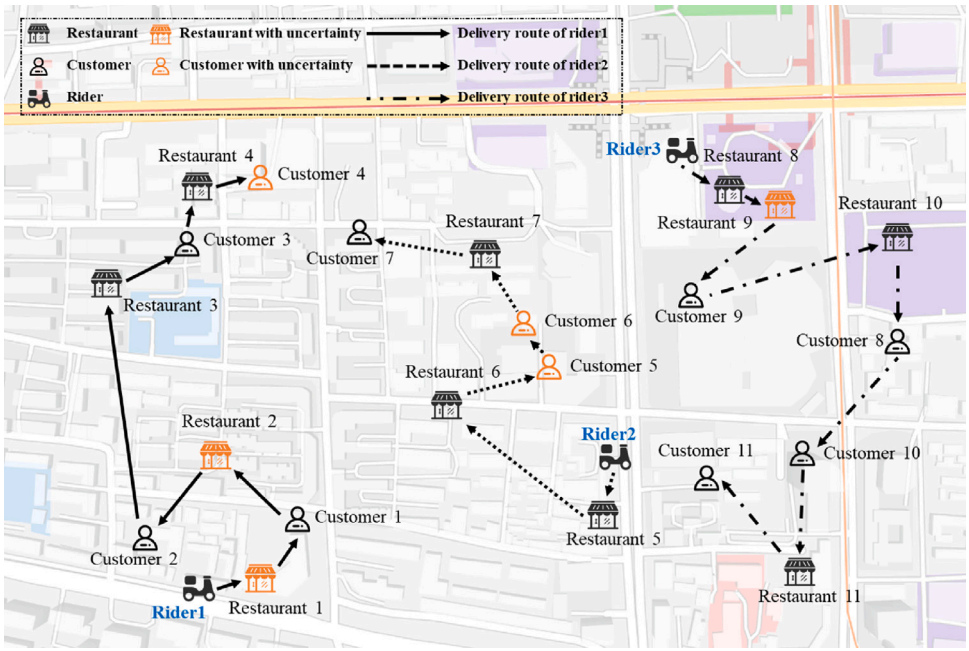



**Fig. 1.** Illustration of food delivery routes.

rider allocation and route planning based on deterministic assumptions may lead to fixed rider loads that are insufficient to accommodate demand variability, potentially causing delays in order fulfillment and affecting subsequent deliveries. Rider service times are similarly uncertain; if riders cannot meet expected delivery times, delay costs may be incurred, and customer satisfaction may decrease. Therefore, developing effective path optimization methods that account for uncertainty is crucial for the future growth of the food delivery industry.

To address these operational uncertainties, current research frequently employs techniques such as stochastic optimization (Li et al., 2010) and robust optimization (Bertsimas & Sim, 2003; Wu et al., 2017) to address these uncertainties. Stochastic optimization typically assumes that random variables follow a specified probability distribution, aiming to find an optimal or near-optimal solution under uncertain conditions (Powell, 2019). However, obtaining accurate distributional information for random variables in practice is often challenging. When randomness becomes highly complex, such that even the distribution type is indeterminate, computational complexity increases significantly, making feasible solution generation much more difficult (Li & Liu, 2009). Compared to stochastic optimization, robust optimization offers notable advantages. Robust optimization models parameter uncertainty by utilizing boundary characteristics of uncertain data, thereby mitigating the limitations of stochastic methods, which rely heavily on prior knowledge and probability distribution assumptions (Gabrel et al., 2014).

In recent years, machine learning-based algorithms have gained significant attention for solving challenging optimization problems (Liang et al., 2022; Lu et al., 2025, 2024). Among these, reinforcement learning has emerged as a promising approach for guiding the optimization process. By utilizing historical optimization solutions as "knowledge", it enhances the efficiency of the search process and improves optimization performance through iterative learning. Q-learning, a value-based learning method in reinforcement learning, learns optimization strategies by interacting with its environment (Chen & Wang, 2024). The integration of Q-learning with heuristic algorithms presents a novel framework for solving routing optimization problems, offering an effective approach to addressing these challenges.

Given this background, this study aims to develop a cost-effective delivery plan robust to uncertainties in both customer demand levels and service times. The main contributions of this paper are as follows: (1) A practical and comprehensive model is developed for the food delivery routing problem. This model incorporates two critical uncertainty factors—customer demand and service time—into the optimization process, thereby enhancing its robustness and adaptability to uncertain environments. (2) A large neighborhood search algorithm based on the Q-learning strategy is proposed to solve large-scale instances. The integration of Q-learning mechanism significantly strengthens the algorithm's performance. (3) Comprehensive experimental validation is conducted using datasets generated from benchmark instances in previous studies. The performance of the proposed algorithm is evaluated by comparing it with CPLEX and adaptive large neighborhood search across instances of varying scales. The experimental results demonstrate the significant superiority and effectiveness of the proposed algorithm in solving the food delivery routing problem.

The remainder of this paper is organized as follows: Section 2 provides a literature review, summarizing prior work in this field. Section 3 details the research problem and presents a mathematical model to capture its characteristics. Section 4 explains the specifics and implementation of the proposed algorithm. Section 5 validates the algorithm through extensive case studies, with a comprehensive analysis and discussion of the results. Finally, Section 6 concludes the paper by offering managerial insights and recommendations.

## 2. Literature review

With the continuous rise in urban on-demand delivery services, the Food Delivery Routing Problem (FDRP) has emerged as a significant topic within the field of operations research and optimization. Essentially, the FDRP can be regarded as a specialized form of the Pickup and Delivery Problem with Time Windows (PDPTW), which itself is a key variant of the Vehicle Routing Problem with Time Windows (VRPTW).

As a fundamental problem in logistics and transportation optimization, the Vehicle Routing Problem (VRP) encompasses a wide range of variants, among which VRPTW and PDPTW are particularly prominent due to their complexity and practical relevance. Readers are referred to recent comprehensive surveys by Elshaer and Awad (2020), Liu et al. (2023), Salehi Sarbijan and Behnamian (2023) and Endler et al. (2023) for an in-depth understanding of the latest developments in VRP research. In VRPTW, customers are served within specified time windows, and violations are typically modeled using penalty mechanisms

for both hard and soft constraints (Taş et al., 2014). PDPTW extends this framework by introducing precedence constraints between pickups and deliveries, further increasing problem complexity and applicability in scenarios involving multiple depots, diverse customer types, and heterogeneous demands (Berbeglia et al., 2007; Koç et al., 2020; Lyu & Yu, 2023). Research perspectives on PDPTW vary depending on the application context. For instance, Yang and Li (2025) focused on last-mile delivery under the omnichannel retailing paradigm for third-party logistics providers. Li et al. (2015) proposed an iterative local search algorithm to address pickup and delivery optimization in multi-depot systems, improving solution quality and computational performance under real-world logistics conditions.

In the context of FDRP, Yildiz and Savelsbergh (2019) developed a novel model under the assumption of full order information and applied a joint column-and-row generation algorithm that demonstrated significant benefits in order consolidation, rider scheduling, and demand management. Other studies have incorporated multi-objective and sustainable delivery considerations. Liao et al. (2020) formulated a tri-objective model integrating carbon footprint minimization, customer satisfaction, and rider fairness, and solved it using a hybrid framework combining Nondominated Sorting Genetic Algorithm II and Adaptive Large Neighborhood Search (ALNS). Similarly, Liu et al. (2024) proposed a rolling-horizon-based hybrid fleet scheduling method, simultaneously optimizing carbon emissions and operational cost. In the area of multimodal delivery, Hamid et al. (2023) integrated crowd-sourced couriers and unmanned aerial vehicles into a collaborative delivery system, proposing an adaptive hyper-heuristic algorithm to solve a multi-depot scheduling model with customer satisfaction constraints. Similarly, Liu (2019) introduced a real-time UAV dispatching framework for high-frequency urban delivery requests, leveraging a rolling-horizon approach and heuristic methods to address the dynamic and uncertain nature of the delivery environment.

Given that routing optimization problems are NP-hard and characterized by high computational complexity, researchers have developed a wide range of solution methods. Exact algorithms, including Branch-and-Bound (Ghilas et al., 2018) and Branch-and-Price (Liberatore et al., 2011), are effective for small-scale instances. For example, Yildiz and Savelsbergh (2019) employed a combined column-and-row generation approach to solve the food delivery routing problem. However, heuristic methods demonstrate superior performance for large-scale problems due to their enhanced scalability and computational efficiency. These approaches include Genetic Algorithms (Liu et al., 2013), Simulated Annealing (Wang et al., 2015), Variable Neighborhood Search (Liu & Jiang, 2022), and ALNS (Hornstra et al., 2020). Wu et al. (2023) proposed a hybrid ant colony optimization algorithm that integrates adaptive parameter tuning and pheromone strategies to enhance performance. Liu et al. (2019) and Brandão (2018) utilized ALNS and Iterated Local Search, respectively, both demonstrating high solution quality and robustness. Within the specific context of online food delivery on platforms such as Meituan in China, Chen, Wang et al. (2022) developed an efficient matching algorithm called Matching Algorithm with Arrival Time Synchronization. Additionally, Feng and Wei (2023) proposed an enhanced multi-directional local search method to approximate the Pareto front for multi-objective optimization problems.

To better reflect the complexity of real-world delivery environments, various uncertainty factors have been progressively incorporated into routing models. These include stochastic customer demands (Zhen et al., 2025) and stochastic travel times (Zhang et al., 2012). In response, researchers have proposed diverse modeling and optimization strategies tailored to different sources of uncertainty. For instance, to address uncertainties in order preparation and customer arrival times, Ulmer et al. (2021) introduced an anticipated customer assignment policy, which leverages delayed assignment and scheduling buffers to enhance system resilience and responsiveness. In large-scale online scheduling scenarios, Zheng, Wang, Wang, Wang et al. (2022) focused on variability in food preparation times and proposed an iterative greedy algorithm combined with task decomposition to mitigate the challenges of search space explosion and risk accumulation. In dealing with dynamic demand and order cancellation probabilities, Xue et al. (2025) formulated a two-stage stochastic model and developed a parallel ALNS algorithm for robust optimization. Regarding uncertainties in delivery time and service processes, Huang et al. (2021) proposed a cluster-driven dynamic scheduling framework for drone delivery systems, incorporating simulated annealing and roulette wheel selection to achieve efficient and stable scheduling under disturbances in flight range and timing. When service time distributions are difficult to estimate, Zheng, Wang, Wang, Chen et al. (2022) designed a hybrid estimation of distribution algorithm integrating an encoding–decoding mechanism with an Expectation–Maximization-based local intensification strategy, significantly improving the accuracy and predictive performance of temporal distribution modeling. Furthermore, Xue et al. (2024) addressed highly coupled cooking and delivery time scenarios by proposing an island-based Harmony Search algorithm that balances solution quality and computational efficiency under uncertainty-dominated environments.

Robust optimization is frequently applied to handle stochastic factors such as travel time uncertainty in VRP (Agra et al., 2013). Wu et al. (2017) proposed a weak robust optimization model aimed at mitigating the deterioration of the objective function, comparing this new robustness criterion with classical robustness criteria, such as best-case, worst-case, and min–max deviation. As long as the uncertainty set is properly designed, consistency between the robust optimization problem and the original problem can be maintained, preserving tractability (Bertsimas & Sim, 2003). Souyris et al. (2013) introduced a robust optimization model for VRP with soft time windows and uncertain service times, solved using a branch-and-price approach. Wang et al. (2022) further explored the integration of branch-price-and-cut solvers with the robust optimization framework to address parametric uncertainty in vehicle routing problems.

Reinforcement learning (RL), with its capability to adaptively learn policies through interaction with dynamic environments, is emerging as a powerful approach to solve uncertain vehicle routing problems. It has shown superior generalization and real-time adaptability compared to traditional methods. Early research primarily addressed uncertainties arising from fluctuating demand and dynamic route conditions. Jahanshahi et al. (2022) applied various Deep Q-Network (DQN) variants, such as DDQN and DDQN-PER, to optimize delivery profitability and service responsiveness under uncertain order arrival times and delivery paths in instant food delivery contexts. Building on this, Chen, Ulmer et al. (2022) extended DQN frameworks to collaborative vehicle–drone same-day delivery systems, reducing action space and incorporating dynamic uncertainties in vehicle availability and customer location, thereby improving overall scheduling performance.

RL also demonstrates flexible modeling capabilities in handling uncertainties related to route resources and vehicle states. Lin et al. (2021) proposed an end-to-end policy gradient method that integrates attention mechanisms and graph embedding modules to handle stochastic disturbances in electric vehicle routing, including time windows, energy consumption, and recharging strategies. Li, Xin et al. (2022) focused on pickup-and-delivery problems with sequential and pairing constraints and introduced a heterogeneous attention-based policy network that exhibited strong robustness against demand fluctuations and route variations, making it suitable for real-world dynamic settings. As uncertainty scenarios grow increasingly complex, hybrid algorithmic frameworks have become an important direction to enhance solution capability. Qin et al. (2021) proposed an RL-driven hyper-heuristic framework based on a distributed Proximal Policy Optimization algorithm. The system dynamically orchestrates metaheuristics such as Artificial Bee Colony and Genetic Algorithms to handle dual uncertainties in heterogeneous vehicles and evolving customer demands. In multimodal transport networks, Zhang et al. (2024) tackled travel

time uncertainty characterized by right-skewed distributions using a weighted multi-objective Q-learning approach. Their model effectively balanced carbon emissions, cost, and delivery time, demonstrating the potential of RL in handling complex multi-objective optimization under uncertainty.

A review of the relevant literature reveals that most studies on food delivery are based on deterministic models, and those that consider uncertainty typically address only a single uncertain factor. This limitation restricts the model's ability to fully capture the impact of multiple uncertainties on the delivery process. Although existing reinforcement learning methods demonstrate good adaptability in addressing uncertainty factors, they still face issues such as reliance on predefined perturbation operators, lack of structural guidance, and insufficient synergy between local and global optimization. These challenges often lead to decreased search efficiency and solution quality when dealing with complex scenarios and large-scale instances. Therefore, this paper constructs a robust version of the meal delivery routing optimization model, considering customer demand uncertainty and service time uncertainty. It proposes the QL-LNS algorithm, which integrates Q-learning with Large Neighborhood Search. By dynamically selecting perturbation operators through reinforcement learning, the algorithm intelligently guides the search process, thereby enhancing its solving capability and stability in uncertain environments. This research not only enhances the theoretical foundation of food delivery route optimization but also offers practical guidance for online service platforms in managing route planning under uncertain conditions.

## 3. Problem description and mathematical formulation

### 3.1. Problem description

This problem can be represented as a network $G = \{N, A\}$, where $N$ is the set of nodes and $A$ is the set of arcs. The node set $N$ consists of the depot node $\{0, 2n+1\}$, which serves as the distribution center, the set $P$ of restaurant nodes representing pickup locations where riders collect orders, and the set $D$ of customer nodes representing delivery destinations where orders are completed upon arrival. The set $A = \{(i,j)\,|i \in N, j \in N\}$ encompasses all possible paths within the network.

In planning delivery routes, specific constraints must be met. First, each order's restaurant node and corresponding customer node must be on the same route to ensure delivery continuity. Additionally, riders are required to visit the restaurant node first to complete the pickup before proceeding to the respective customer node for delivery.

In a food delivery platform, there are $n$ orders that must be assigned to $|K|$ available riders. Each node $i \in P \cup D$ is associated with an order and has a demand $r_i$ ($r_i > 0$ for $i \in P$, $r_i < 0$ for $i \in D$). Riders typically use electric bikes, which imposes a vehicle load limit $L$ to ensure safety and efficiency. Each node $i$ is associated with a service time window $[\alpha_i, \beta_i]$, where $\alpha_i$ and $\beta_i$ denote the earliest and latest allowable arrival times, respectively. For customer nodes $D$, $\beta_i$ represents the latest acceptable delivery time; if a rider arrives at node $i$ after $\beta_i$, the delivery is considered late. At restaurant nodes $P$, $\alpha_i$ indicates the meal preparation time; if a rider arrives earlier than $\alpha_i$, they must wait until $\alpha_i$ to collect the order. The travel distance between nodes $i$ and $j$ is denoted as $d_{ij}$, with an associated travel time $t_{ij}$. Upon arrival at node $i$, riders may need to walk a short distance to complete the pickup or delivery, which requires an additional service time $t_s$. Each rider operates within a predefined work window $[e_k, l_k]$, outside of which they cannot accept new orders. If a rider has not completed their assigned orders by $l_k$, they will continue until all deliveries are completed, ensuring that all tasks are fulfilled before ending their shift. The objective of this problem is to minimize delivery costs while ensuring customer satisfaction. To achieve this goal, the model is based on the following assumptions:

(1). All riders depart from a single distribution center and return there after completing all assigned deliveries.

**Table 1**
Summarizes the notations used in the mathematical model.

| | |
|---|---|
| **Indices and sets** | |
| $i, j$ | Index of nodes |
| $k$ | Index of riders |
| $N$ | Set of all nodes, $N = \{0, 2n+1\} \cup P \cup D$ |
| $K$ | Set of all riders |
| **Parameters** | |
| $f_1$ | Rider activation costs, including rider base wages and electric bike maintenance costs |
| $f_2$ | Distribution cost per unit distance |
| $d_{ij}$ | Distance between node $i$ and node $j$ |
| $\alpha_i$ | Earliest time in node $i$'s service time window |
| $\beta_i$ | The latest time in the service time window for node $i$ |
| $r_i$ | The level of demand to be loaded at node $i$, positive if it is a pickup and negative if it is a delivery |
| $L$ | Maximum load limit for riders |
| $t_{ij}$ | Traveling time between node $i$ and node $j$ |
| $t_i^s$ | Riders' service time at the node $i$ |
| $e_k$ | The earliest start time for rider $k$, and rider $k$ can only accept orders placed at a time greater than or equal to this time |
| $l_k$ | The latest off-duty time for rider $k$, if the time beyond rider cannot take orders, but will continue to deliver the orders received before the completion |
| **Decision variables** | |
| $x_{ij}^k$ | Binary variable, equals 1 if rider $k$ travels from node $i$ to node $j$, otherwise 0 |
| $y_i^k$ | Actual loading of rider $k$ at node $i$ |
| $\tau_i^k$ | The time at which rider $k$ reaches node $i$ |

(2). Each customer is served by only one rider, meaning that each order is assigned to a single rider.

(3). Riders travel at a constant speed throughout the delivery process, unaffected by external factors.

(4). The service time required by a rider for each customer is known, uniform, and fixed.

(5). The demand for each order is fixed and known in advance.

### 3.2. Deterministic model

The definitions of all variables mentioned in this section are provided in Table 1. Based on the above assumptions, the deterministic model is formulated as follows:

$$[\mathrm{M}_0] \quad \min \quad f_1 \sum_{j \in N \setminus \{2n+1\}} \sum_{k \in K} x_{0j}^k \quad + \quad f_2 \sum_{i \in N} \sum_{j \in N} \sum_{k \in K} x_{ij}^k d_{ij}, \tag{1}$$

subject to:

$$\sum_{k \in K} \sum_{j \in P \bigcup D} x_{ij}^k = 1 \qquad \forall i \in P, \tag{2}$$

$$\sum_{j \in N} x_{ij}^k - \sum_{j \in N} x_{j,i+1}^k = 0 \qquad \forall i \in P, k \in K, \tag{3}$$

$$\sum_{j \in P \bigcup \{2n+1\}} x_{0j}^k = 1 \qquad \forall k \in K, \tag{4}$$

$$\sum_{i \in D \bigcup \{0\}} x_{i,2n+1}^k = 1 \qquad \forall k \in K, \tag{5}$$

$$\sum_{i \in N} x_{ij}^k - \sum_{i \in N} x_{ji}^k = 0 \qquad \forall j \in P \cup D, k \in K, \tag{6}$$

$$\tau_i^k + t_{ij} + t_i^s - \tau_j^k \le (1 - M) x_{ij}^k \qquad \forall i, j \in N, k \in K, \tag{7}$$

$$\alpha_i \le \tau_i^k \le \beta_i \qquad \forall i \in N, k \in K, \tag{8}$$

$$\tau_i^k \le \tau_{i+1}^k \qquad \forall i \in P, k \in K, \tag{9}$$

$$y_i^k + r_j - y_j^k \le (1 - M) x_{ij}^k \qquad \forall i, j \in N, k \in K, \tag{10}$$

$$y_i^k \le L \qquad \forall i \in N, k \in K, \tag{11}$$

$$y_0^k = y_{2n+1}^k = 0 \qquad \forall k \in K, \tag{12}$$

$$\alpha_i - l_k + (\sum_{j \in P \bigcup D} x_{ij}^k - 1)M \le 0 \qquad \forall i \in P, k \in K, \tag{13}$$

$$\alpha_i - e_k + (1 - \sum_{j \in P \bigcup D} x_{ij}^k)M \ge 0 \qquad \forall i \in P, k \in K, \tag{14}$$

$$x_{ij}^k \in \{0,1\} \qquad \forall i, j \in N, k \in K, \tag{15}$$

$$\tau_i^k \ge 0 \qquad \forall i \in N, k \in K, \tag{16}$$

$$y_i^k \ge 0 \qquad \forall i \in N, k \in K. \tag{17}$$

The objective function aims to minimize the total cost of the delivery plan, which comprises both the fixed activation costs of riders and variable delivery costs. Constraint (2) ensures that each order is assigned to and completed by a single rider, guaranteeing full order allocation and completion. Constraint (3) enforces that both pickup and delivery nodes for each order are served by the same rider, maintaining order continuity and consistency. Constraints (4) and (5) require that each rider departs from and ultimately returns to the distribution center. Constraint (6) mandates that all nodes, except the distribution center, must be visited by a rider, ensuring that every node is included in a rider's route. Constraints (7), (8), and (9) precisely calculate the arrival times of riders, ensuring sequential arrival at each node within the specified time windows. Constraints (10), (11), and (12) regulate rider load levels, restricting them to a maximum load L during deliveries. Constraints (13) and (14) stipulate that order assignments and deliveries must be completed within each rider's working time window, ensuring task completion within scheduled hours. Finally, Constraints (15), (16), and (17) define the allowable ranges for the decision variables.

### 3.3. Robust model considering demand and service time uncertainties

In Section 3.2, we introduced a deterministic model. However, in real-world food delivery operations, numerous random factors often affect delivery conditions, making them far more complex than those captured by a deterministic model. Specifically, Assumptions 4 and 5 treat service time and order demand as fixed, constant factors. While this simplification reduces model complexity, it may also create discrepancies between the optimized solution and real-world conditions. To enhance the practical applicability of the optimization solution, this study proposes a robust food delivery route optimization model based on the deterministic framework. This robust model accounts for random fluctuations in service times and order demand, addressing the unpredictability commonly encountered in practice. Our goal is to develop an optimization solution with greater robustness and capable of effective operation under uncertain conditions.

Bertsimas and Sim (2004) introduced a robust optimization approach based on a budgeted uncertainty set, which has been widely adopted and extensively studied in the literature (Fowler & Mönch, 2022; Li, Zhang et al., 2022; Ning & You, 2019). The goal of robust optimization is to identify a solution that remains feasible under all possible realizations of uncertain data. Specifically, uncertainty in the data is represented by an uncertainty set that encompasses all potential scenarios for the random variables. To balance robustness with avoiding overly conservative solutions, Bertsimas and Sim introduced a robust budget parameter, $\Gamma$, which allows for control over the level of robustness in the model. This method reformulates the original model into a robust counterpart (RC) that can be solved directly. The budgeted uncertainty set, a classic and widely used approach, can be considered a specific case of the box uncertainty set and holds significant value in handling robust optimization problems with uncertain data (Cheramin et al., 2021).

In practical delivery operations, the degree of uncertainty often varies across different routes. Certain routes may be more susceptible to uncertainties, while others remain relatively stable. Additionally, since each rider handles a different number of orders, those serving more customers are likely to encounter higher levels of uncertainty. Therefore, it is essential to impose limits on the maximum level of uncertainty for each route to better control the robustness of the delivery plan (Hu et al., 2018; Lee et al., 2012). Based on this discussion, we define the uncertainty sets for customer demand levels and service times.

In Eq. (18), $u_t$ is the overall service time uncertainty set, and $U_t^k$ is the Cartesian product of the service time uncertainty set for each rider.

$$u_t = \times_{k \in K} U_t^k, \tag{18}$$

with:

$$U_t^k = \left\{ \tilde{t} \in R^{|A^k|} | \tilde{t}_i = \bar{t}_i + \upsilon_i \hat{t}_i, \sum_{i \in A^k} |\upsilon_i| \le \Gamma_k, 0 \le \upsilon_i \le 1, \forall i \in A^k \right\}, \tag{19}$$

$$\Gamma_k = \left\lceil \gamma_t |A^k| \right\rceil. \tag{20}$$

In Eq. (19), $U_t^k$ represents the uncertainty set model for the service time of each rider $k$. The set $A^k$ denotes the collection of nodes that rider $k$ is assigned to serve. For each node $i \in A^k$, service time uncertainty arises from the values within $[\bar{t}_i - \hat{t}_i, \bar{t}_i + \hat{t}_i]$, where $\bar{t}_i$ is the nominal value of the uncertain service time, and $\hat{t}_i$ is the maximum deviation from this nominal value. The auxiliary variable $\upsilon_i$ captures the extent of deviation of the actual service time from this value. In Eq. (20), $\Gamma_k$ represents the uncertainty budget for rider $k$ within the service node set, determined by parameters $\gamma_t$ and $|A^k|$. Here, $\gamma_t$ is the uncertainty budget coefficient for service time, and $|A^k|$ is the number of nodes rider $k$ serves. To ensure that the uncertainty budget is an integer, $\lceil \gamma_t |A^k| \rceil$ denotes the smallest integer greater than or equal to $\gamma_t |A^k|$. The parameter $\gamma_t$ ranges between [0,1]; when $\Gamma_k$=0, it indicates that service times at all nodes are deterministic, with no deviation from nominal values.

In Eq. (21), $u_r$ is the overall demand uncertainty set and $U_r^k$ is the Cartesian product of the service time uncertainty set for each vehicle.

$$u_r = \times_{k \in K} U_r^k, \tag{21}$$

with:

$$U_r^k = \left\{ \tilde{r} \in R^{|A^k|} | \tilde{r}_i = \bar{r}_i + o_i \hat{r}_i, \sum_{i \in A^k} |o_i| \le \Lambda_k, 0 \le o_i \le 1, \forall i \in A^k \right\}, \tag{22}$$

$$\Lambda_k = \gamma_r |A^k|. \tag{23}$$

Similar to the interpretation of service time uncertainty parameters, the uncertain demand at each node $i \in A^k$ is drawn from values within the set $[\bar{r}_i - \hat{r}_i, \bar{r}_i + \hat{r}_i]$, where $\bar{r}_i$ is the nominal value of uncertain demand and $\hat{r}_i$ represents the maximum deviation from this nominal demand. Additionally, the auxiliary variable $o_i$ is used to capture the degree of deviation from the nominal demand value. In Eq. (23), $\gamma_r$ denotes the demand uncertainty budget, which constrains the level of demand uncertainty that a rider can handle across their assigned node set. Ranging from [0,1], $\Lambda_k$ determines the extent of uncertainty permissible in the rider's service.

Based on the above-defined uncertainty sets, we can extend the previous deterministic model into a robust optimization model. In the robust optimization framework, only the extreme cases of uncertainty need to be considered, specifically when $\upsilon_i = o_i = 1$. Following the principles of adjustable robust optimization, certain decision variables can be adjusted within the predefined range of uncertain parameters (Agra et al., 2013; Hu et al., 2018). Therefore, the decision variables $\tau_i^k$ and $y_i^k$ in the previous model are modified into adjustable decision variables

**Table 2**
Illustrative example.

| Node | Time window | Outgoing arcs and travel times |
|---|---|---|
| 0 | [1, 100] | (0,1) : 6; (0,2) : 21; (0,3) : 17; (0,4) : 30 |
| 1 | [0, 15] | (1,2) : 15; (1,3) : 11; (1,4) : 24 |
| 2 | [13, 27] | (2,3) : 4; (2,4) : 9 |
| 3 | [17, 35] | (3,4) : 13 |
| 4 | [30, 50] | — |

$\tau_i^k(t,r)$ and $y_i^k(t,r)$, where $t \in u_t$ and $r \in u_r$. The new robust model is formulated as follows:

$$[\mathrm{M}_1] \quad \min \quad f_1 \sum_{j\in N, j\neq 2n+1} \sum_{k\in K} x_{0j}^k + f_2 \sum_{i\in N} \sum_{j\in N} \sum_{k\in K} x_{ij}^k d_{ij}, \tag{24}$$

subject to : (2)–(6), (13)–(15) and

$$\tau_i^k(t,r) + t_{ij} + t_i^s - \tau_j^k(t,r) \le (1-M)x_{ij}^k \qquad \forall i,j \in N, k \in K, t \in u_t, r \in u_r, \tag{25}$$

$$\alpha_i \le \tau_i^k(t,r) \le \beta_i \qquad \forall i \in N, k \in K, t \in u_t, r \in u_r, \tag{26}$$

$$\tau_i^k(t,r) \le \tau_{i+1}^k(t,r) \qquad \forall i \in P, k \in K, t \in u_t, r \in u_r, \tag{27}$$

$$y_i^k(t,r) + r_j - y_j^k(t,r) \le (1-M)x_{ij}^k \qquad \forall i,j \in N, k \in K, t \in u_t, r \in u_r, \tag{28}$$

$$y_i^k(t,r) \le L \qquad \forall i \in N, k \in K, t \in u_t, r \in u_r, \tag{29}$$

$$y_0^k(t,r) = y_{2n+1}^k(t,r) = 0 \qquad \forall k \in K, t \in u_t, r \in u_r, \tag{30}$$

$$\tau_i^k(t,r) \ge 0 \qquad \forall i \in N, k \in K, t \in u_t, r \in u_r, \tag{31}$$

$$y_i^k(t,r) \ge 0 \qquad \forall i \in N, k \in K, t \in u_t, r \in u_r. \tag{32}$$

In the model restructuring, Eqs. (7)–(12) from the original deterministic model are rewritten as Eqs. (25)–(30) to accommodate the requirements of robust optimization. Similarly, Eqs. (16) and (17) are reformulated as Eqs. (31) and (32).

### 3.4. An illustrative example

To illustrate how uncertainty affects delivery route planning, we build a small instance consisting of a depot (node 0) and four task nodes {1, 2, 3, 4}, where odd-numbered nodes represent restaurants and even-numbered nodes represent customers. The time windows assigned to each node and the pairwise travel times are summarized in Table 2. For example, the rider must arrive at node 1 within the time window [0, 15], and traveling from node 0 to node 1 requires 6 min.

The parameter settings are as follows: unit travel cost is 0.5, unit overtime cost is 4, unit overload cost is 5, and the rider's load capacity limit is 10 units. Based on these data, we generate both a deterministic route and a robust route. The deterministic route is $0 \to 1 \to 3 \to 2 \to 4 \to 0$, with a total travel time of $6 + 11 + 4 + 9 + 30 = 60$ and a cost of 30. The robust route is $0 \to 1 \to 2 \to 3 \to 4 \to 0$, with a total travel time of $6 + 15 + 4 + 13 + 30 = 68$ and a cost of 34.

Fig. 2 compares the two solutions under both deterministic and uncertain service times, with details provided in Table 3. In the deterministic scenario, each node's service time is assumed to be three minutes, and the rider's estimated arrival times satisfy all time window constraints. In the uncertain scenario the actual service time at node 3 increases to five minutes. As reported in Table 3, this extra two minutes causes the rider to reach node 2 at time 29, which exceeds the time window by two minutes. Consequently, an overtime penalty of $4 \times 2 = 8$ would be incurred, and the total cost of the deterministic route increases to 38, which is higher than the cost of the robust route. In contrast, the robust solution demonstrates stronger adaptability. As reported in Table 3, even with the prolonged service time at node 3, the robust solution maintains sufficient buffer times. The rider consistently arrives within the time windows of subsequent nodes, and the total cost remains at 34.

Table 3 also compares rider's load under different demand scenarios. Under the deterministic demand, the deterministic solution avoids overload, although the load at node 3 is very close to the capacity limit. If node 3's demand increases to seven units, the rider's load exceeds the limit, resulting in an overload penalty of $(5 \times 2 = 10)$. Conversely, the robust route keeps the load within the capacity limit under both the expected and the increased demand, thus avoiding any overload penalty.

This illustrative example clearly demonstrates that uncertainty can disrupt the planned routes and degrade service levels. While the deterministic solution achieves lower costs under expected conditions, its resilience to uncertainty is limited, leading to higher overall costs when overtime or overload penalties occur. In contrast, the robust solution, despite incurring slightly higher initial costs, effectively mitigates the negative impacts of uncertainty, ensures better delivery stability. Given that uncertainty cannot be fully anticipated during planning, route robustness is critical. By reserving adequate buffer times and capacity in the route design, potential service delays or demand fluctuations can be accommodated more effectively, ultimately enhancing service levels and controlling overall operational costs.

## 4. Design of the algorithm

### 4.1. Basic structure of QL-LNS

The QL-LNS algorithm incorporates a reinforcement learning mechanism into the traditional Large Neighborhood Search (LNS) framework. By dynamically adjusting the selection strategy of destroy and repair operators, it enables efficient exploration of the solution space and adaptive optimization. The core concept of QL-LNS lies in embedding the Q-learning mechanism into the search process, allowing the algorithm to iteratively refine the combination strategies of operators based on feedback from the search results. This approach improves both search efficiency and solution quality. In contrast to traditional LNS methods that depend on predefined or heuristic rules for operator selection, QL-LNS constructs a Q-value table to learn the effects of various operator combinations on search performance through iterative learning. It prioritizes combinations that have historically led to better improvements, thereby avoiding local optima and enhancing the algorithm's convergence performance and stability.

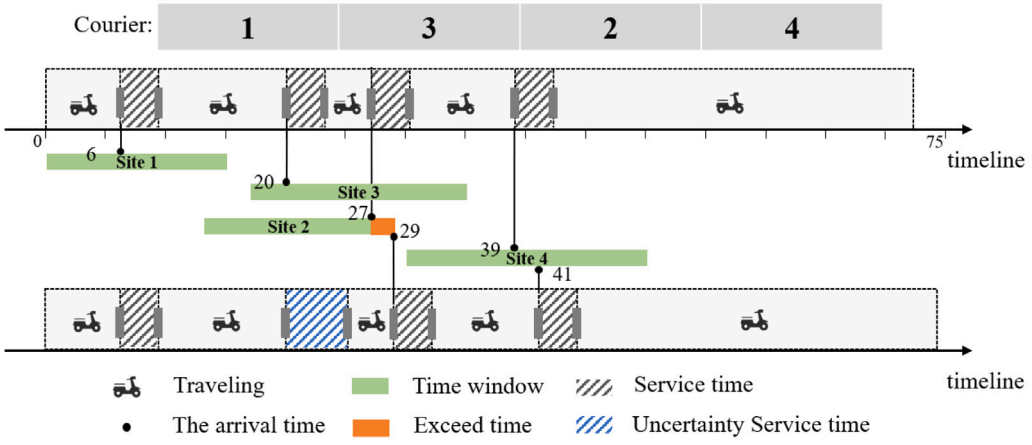
The QL-LNS algorithm operates through the following four main stages: (1) Initialization: Generate the initial solution and initialize the Q-table; (2) Operator Selection: Select a destroy–repair operator combination based on the Q-value table and an exploration–exploitation strategy; (3) Perturbation and Reconstruction: Apply the selected operators to destroy and repair the current solution; (4) Reward Evaluation and Q-value Update: Update the Q-table based on the quality of the new solution. This iterative process continues until a predefined termination criterion is met. The definitions of states, actions, rewards, and the Q-table within the Q-learning framework are detailed as below.

**State**: In QL-LNS, each state represents a specific destruction–repair operator combination. For example, given three destruction operators (D1, D2, D3) and three repair operators (R1, R2, R3), the total number of state combinations is nine, expressed as: $S = \{(D_i, R_j) \mid i = 1,2,3; j = 1,2,3\}$. Each state $s = (D_i, R_j)$ corresponds to the application of the destruction operator $D_i$ and repair operator $R_j$ to destroy and repair the current solution. Detailed explanations of each operator are provided in Section 4.2.

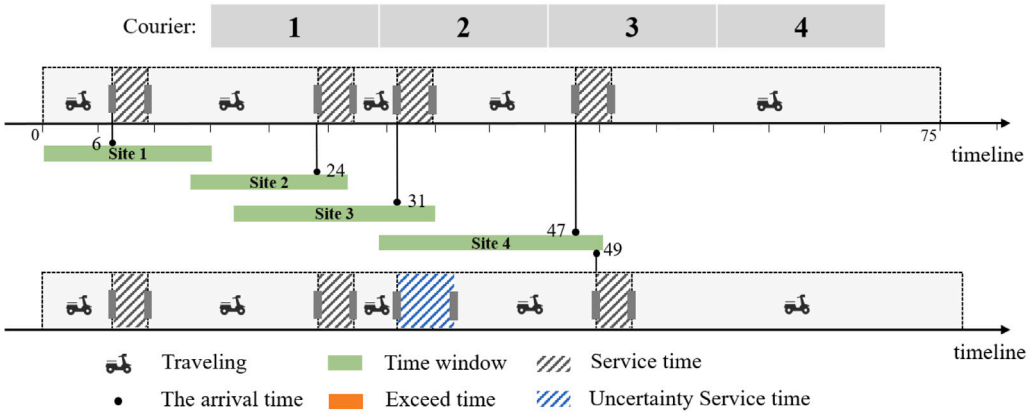
**Action**: An action is defined as a transition from the current state (a specific operator combination) $s = (D_i, R_j)$ to another state $s' = (D_{i'}, R_{j'})$, representing a switch to a new operator combination. The

**Table 3**
Illustrative example: Comparison of deterministic and robust solutions.

| Type | Site | Uncertain service time | | | | | Uncertain demand | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Expected time | | Actual time | | Exceed time | Expected | | Actual | | Exceed load |
| | | Service | Arrival | Service | Arrival | | Demand | Load | Demand | Load | |
| Det | 1 | 3 | 6 | 3 | 6 | 0 | 5 | 5 | 5 | 5 | 0 |
| | 3 | 3 | 20 | 5 | 20 | 0 | 5 | 10 | 7 | 12 | 2 |
| | 2 | 3 | 27 | 3 | 29 | 2 | −5 | 5 | −5 | 7 | 0 |
| | 4 | 3 | 39 | 3 | 41 | 0 | −5 | 0 | −7 | 0 | 0 |
| Rob | 1 | 3 | 6 | 3 | 6 | 0 | 5 | 5 | 5 | 5 | 0 |
| | 2 | 3 | 24 | 3 | 24 | 0 | −5 | 0 | −5 | 0 | 0 |
| | 3 | 3 | 31 | 5 | 31 | 0 | 5 | 5 | 7 | 7 | 0 |
| | 4 | 3 | 47 | 3 | 49 | 0 | −5 | 0 | −7 | 0 | 0 |




(a) The deterministic solution




(b) The robust solution

**Fig. 2.** Illustrative example: Gantt chart.

action space includes all valid state transitions, and the execution of an action is reflected in the application of the new operator combination during the next iteration of destruction and repair.

**Reward**: The reward function is to reflect the improvement in the objective function of the solution. Let the current solution be $X_{cur}$, and the new solution after executing an action be $X_{new}$. If the objective function value improves, a positive reward $r = r + C(X_{new})$ is assigned; otherwise, the reward is set to 0.

**Q-table Structure and Update Mechanism**: The Q-table, shown in Table 4, is structured as a 9 × 9 matrix, where the dimension corresponds to the number of states. When the current solution $X_{cur}$ executes an action, leading to an improved solution $X_{new}$, the Q-value is updated using the following formula:

$$Q(s,a) = Q(s,a) + \chi[r + \varsigma \cdot \max_{a'} Q(s',a') - Q(s,a)]. \tag{33}$$

Here, $\chi$ is the learning rate, $\varsigma$ is the discount factor. During the initialization phase, all Q-table entries are set to zero. The Q-value update mechanism is guided by the reward, which is directly related to the degree of improvement in the solution. This design allows the algorithm to adaptively prioritize search paths that yield greater improvements, enhancing both the efficiency and the quality of the solutions.

The specific steps of Q-learning are presented in pseudocode in Algorithm 1, where $X$ and $C(X)$ represent the solution and its corresponding objective value, respectively. The algorithm begins by randomly selecting an initial state $s$, and then determines the action a based on

the selection policy to update the state $s$, continuing until the final state $s'$ is reached. During this process, the solution is iteratively updated based on the current state $s$, and the Q-table is trained and updated according to the obtained reward $r$.

**Algorithm 1:** Framework for training Q-table

**Input:** Parameters: the learning rate $\chi$, discount factor $\varsigma$
Initialize Q-table, random state $s$;
$X_{\text{cur}} \leftarrow X_{\text{best}}, \quad X_{\text{new}} \leftarrow (X_{\text{cur}}, s)$;
**while** $s$ *is not terminal* **do**
  Select action $a$ from $s$ using **Selection Strategy** (Algorithm 3);
  $s' \leftarrow (s, a), \quad X_{\text{new}} \leftarrow (X_{\text{cur}}, s')$;
  **if** $C(X_{new}) < C(X_{cur})$ **then**
    $r \leftarrow r + C(X_{\text{new}})$;
    $Q(s,a) \leftarrow Q(s,a) + \chi\left[r + \varsigma \cdot \max_{a'} Q(s',a') - Q(s,a)\right]$;
  **if** $C(X_{new}) < C(X_{best})$ **then**
    $r \leftarrow r + C(X_{\text{new}}), \quad X_{\text{best}} \leftarrow X_{\text{new}}$;
    $Q(s,a) \leftarrow Q(s,a) + \chi\left[r + \varsigma \cdot \max_{a'} Q(s',a') - Q(s,a)\right]$;
  $s \leftarrow s'$;

Algorithm 2 presents the pseudocode for the QL-LNS algorithm. Initially, the initial solution $X_0$ and Q-table are constructed, with all Q-values set to zero. The algorithm terminates when one of the following conditions is met: the maximum iteration count is reached, or the number of consecutive non-updated iterations exceeds the maximum allowable number of non-updated iterations. If a newly generated solution improves upon the current best solution, the best solution is updated, and the count of non-updated iterations is reset to zero. Additionally, the Q-table is trained every 500 iterations. During each iteration, the algorithm continuously selects actions to update the state $s$ based on the selection policy. If the new solution improves upon the current solution, a reward $r$ is given, and the Q-table is updated. If the new solution does not outperform the current one, the algorithm reselects the state $s$ and modifies the solution again. If the updated solution improves upon the current solution, a reward $r$ is given, and the Q-table is updated. If no improvement is observed, the algorithm employs Simulated Annealing (SA) to decide whether to accept the new solution. The SA process sets an initial temperature $T_0$ and a cooling coefficient of $\lambda$. The current temperature is calculated using the formula $T_c = \lambda T_{last}$. The probability of accepting the current solution $X_{new}$ is given by $p = \exp\left(\left(C\left(X_{cur}\right) - C\left(X_{new}\right)\right)/T_c\right)$. The cooling coefficient $\lambda \in (0,1)$, which gradually reduces the current temperature over time, leading to a decreasing probability of accepting worse solutions. This ensures that in the early stages, the algorithm can explore a larger solution space by accepting worse solutions, even if they do not outperform the previous ones. In the later stages, the acceptance probability decreases, preventing the algorithm from accepting inferior solutions and promoting convergence to a better solution. To quickly find high-quality solutions in a complex search space, the algorithm employs local search to further explore the neighborhood of the updated solution (see Section 4.3 for details).

The pseudocode for the selection strategy is shown in Algorithm 3. Specifically, the algorithm selects an action randomly with probability $\omega$, and with probability $1 - \omega$, it chooses the action corresponding to the maximum value in the Q-table. To enhance understanding of the algorithm's execution process, the corresponding flowchart is provided in Fig. 3.

**Algorithm 2:** Framework of the proposed QL-LNS algorithm

**Input:** Parameters: the acceptable probability $\omega$, initial temperature $T_0$, current temperature $T_c$, cooling coefficient $\lambda$
Initialize $X_0$ and Q-table;
Train Q-table, random state $s$;
$X_{cur} \leftarrow X_0$;
Initialize iteration counter $i \leftarrow 0$, $ite \leftarrow 0$;
**while** $i \le Max_{iter}$ ***or*** $ite \le Max_{no_improved}$ **do**
  **if** $i \bmod 500 = 0$ **then**
    Train Q-table;
  $a \leftarrow SelectionStrategy(Q, s, \omega)$, $s \leftarrow (s, a)$, $X_{new} \leftarrow (X_{cur}, s)$;
  $X_{new} \leftarrow LocalSearch(X_{new})$;
  **if** $C(X_{new}) < C(X_{cur})$ **then**
    **if** $C(X_{new}) < C(X_{best})$ **then**
      $X_{best} \leftarrow X_{new}$, $C(X_{best}) \leftarrow C(X_{new})$;
      $ite \leftarrow 0$;
    **else**
      $ite \leftarrow ite + 1$;
    $X_{cur} \leftarrow X_{new}$, $C(X_{cur}) \leftarrow C(X_{new})$, $r \leftarrow C(X_{new})$;
  **else**
    **while** $C(X_{new}) > C(X_{cur})$ **do**
      $a \leftarrow SelectionStrategy(Q, s, \omega)$, $s \leftarrow (s, a)$, $X_{new} \leftarrow (X_{cur}, s)$;
      $X_{new} \leftarrow LocalSearch(X_{new})$;
      **if** $C(X_{new}) < C(X_{cur})$ **then**
        **if** $C(X_{new}) < C(X_{best})$ **then**
          $X_{best} \leftarrow X_{new}$, $C(X_{best}) \leftarrow C(X_{new})$;
          $ite \leftarrow 0$;
        **else**
          $ite \leftarrow ite + 1$;
        $X_{cur} \leftarrow X_{new}$, $C(X_{cur}) \leftarrow C(X_{new})$, $r \leftarrow r + C(X_{new})$;
        Update Q-table;
      **else**
        $ite \leftarrow ite + 1$;
        $p \leftarrow \exp\left(\frac{C(X_{cur}) - C(X_{new})}{T_c}\right)$;
        $a \leftarrow \text{rand}(1)$;
        **if** $a < p$ **then**
          $X_{cur} \leftarrow X_{new}$, $C(X_{cur}) \leftarrow C(X_{new})$;
  $i \leftarrow i + 1$;

**Algorithm 3:** Selection Strategy

**Input:** Parameters: the acceptance probability $\omega$
**if** *at the beginning of the procedure* **then**
  Randomly select $a \in A$;
**else**
  $\theta \leftarrow \text{rand}(1)$;
  **if** $\theta \le \omega$ **then**
    Randomly select $a \in A$;
  **else**
    Select $a' = \arg\max_{a \in A} Q(s, a)$;

### 4.2. Destroy and repair operators

In delivery route optimization, destruction operators are used to disrupt the structure of the current solution by removing a set of nodes, thereby generating new solution sequences. The destruction process involves deleting specific tasks to modify the original routes, enabling the exploration of alternative paths. This study employs three types of destruction operators: random removal, greedy removal, and related

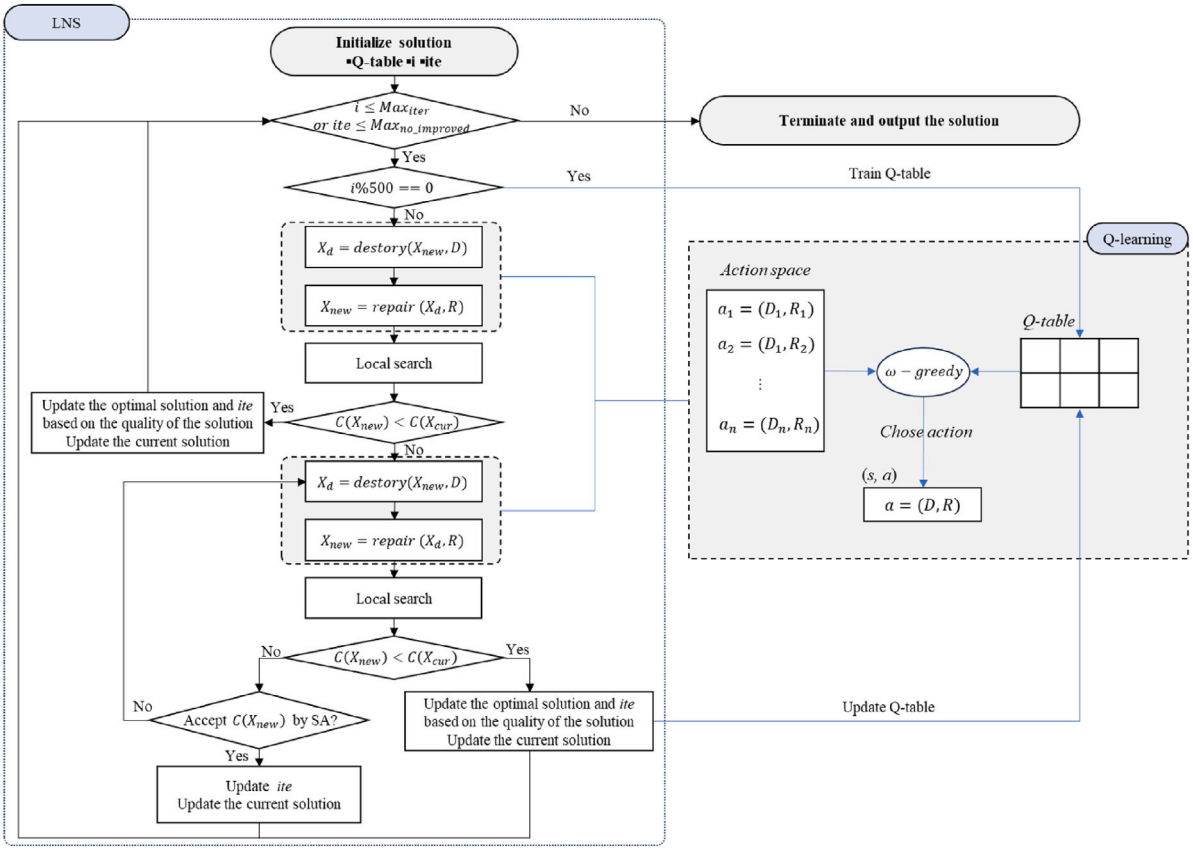



**Fig. 3.** Flow chart of the proposed QL-LNS algorithm.

removal. The removed nodes are stored in a pool of unassigned nodes, which will be reintegrated into the solution during the repair phase.

To ensure the feasibility of the solution, the destruction process adheres to the following guidelines: when removing a node, all related nodes of the same order must also be removed to maintain logical consistency. Since restaurant nodes always precede customer nodes, removing a restaurant node automatically triggers the removal of its corresponding customer node. The extent of destruction, defined as the number of orders to be removed, is a predefined parameter.

The repair process is the inverse of the destruction process, aiming to reintegrate nodes from the unassigned node pool into routes while ensuring all feasibility constraints are satisfied. During repair process, the restaurant node of an order is inserted into the route first, followed by its corresponding customer node. This ensures that related nodes of the same order are placed on the same route and that restaurant nodes are visited before customer nodes in a logical sequence. The algorithm employs three repair operators: random repair, greedy repair, and regret repair. Below are detailed explanations of each operator:

**Random removal**: Random removal is a simple and straightforward destruction heuristic. It randomly deletes nodes from the current delivery solution until a predefined destruction level is reached. By ignoring the cost impact of removing specific nodes on the current solution, this operator introduces randomness into the search process, broadening the exploration of the solution space. This helps prevent the algorithm from prematurely converging to local optima and enhances its ability to identify globally optimal solutions.

**Worst removal**: Worst removal focuses on eliminating nodes that contribute the most to the total cost, thereby improving the overall delivery solution. Each order is temporarily removed, and the resulting cost change is calculated before restoring the route. After evaluating all nodes, the one with the greatest cost reduction is removed and added to the unassigned node pool. If removing a node deactivates a route, the reduction in rider activation cost is also considered. By efficiently identifying and removing nodes with high-cost nodes, this operator significantly enhances solution quality and optimizes the delivery plan.

**Related removal**: Related removal leverages the correlation between closely related orders to optimize route assignments. This correlation is calculated based on distance and time, prioritizing the removal of orders that are highly related. The distance correlation $\Omega_d(q_i, q_j)$ measures the geographical proximity between orders $q_i$ and $q_j$, with smaller distances indicating a higher correlation. The time correlation $\Omega_t(q_i, q_j)$ is determined by the similarity of their service time windows, where closer windows result in a higher correlation. Here, $\eta_1$ and $\eta_2$ represent the weights assigned to the distance and time correlations, respectively. The operator first selects a random order and removes it along with the order that is most closely related to it, adding both to the unassigned node pool. The specific formulas are as follows:

$$\Omega(q_i, q_j) = \eta_1 \Omega_d(q_i, q_j) + \eta_2 \Omega_t(q_i, q_j), \tag{34}$$

where:

$$\Omega_d(q_i, q_j) = d_{2i,2j} + d_{2i,2j-1} + d_{2i-1,2j} + d_{2i-1,2j}, \tag{35}$$

$$\Omega_t(q_i, q_j) = |\alpha_{2i-1} - \alpha_{2j-1}| + |\alpha_{2i} - \alpha_{2j}|. \tag{36}$$

**Random repair**: Random repair reinserts nodes from the unassigned node pool into random positions in the solution. While the insertion positions are chosen randomly, it is ensured that the restaurant and customer nodes of the same order are placed on the same route, with the restaurant node visited first. It should be noted that new route may created as needed. By introducing randomness, this operator expands the search space and reduces the risk of getting trapped in local optima.

**Greedy repair**: The greedy repair operator inserts nodes at positions that result in the smallest cost increase. It evaluates all possible

**Table 4**
Q-table obtained applying Q-learning for the proposed QL-LNS algorithm.

| Action | State | | | | |
|---|---|---|---|---|---|
| | (D1,R1) | (D2,R1) | … | (D2,R3) | (D3,R3) |
| (D1,R1) | $Q((D1,R1),(D1,R1))$ | $Q((D1,R1),(D2,R1))$ | … | $Q((D1,R1),(D2,R3))$ | $Q((D1,R1),(D3,R3))$ |
| (D2,R1) | $Q((D2,R1),(D1,R1))$ | $Q((D2,R1),(D2,R1))$ | … | $Q((D2,R1),(D2,R3))$ | $Q((D2,R1),(D3,R3))$ |
| ⋮ | ⋮ | ⋮ | ⋱ | ⋮ | ⋮ |
| (D2,R3) | $Q((D2,R3),(D1,R1))$ | $Q((D2,R3),(D2,R1))$ | … | $Q((D2,R3),(D2,R3))$ | $Q((D2,R3),(D3,R3))$ |
| (D3,R3) | $Q((D3,R3),(D1,R1))$ | $Q((D3,R3),(D2,R1))$ | … | $Q((D3,R3),(D2,R3))$ | $Q((D3,R3),(D3,R3))$ |

insertion positions for a pair of nodes from the unassigned node pool and selects the position with the minimal cost increase. This process repeats until the unassigned node pool is empty. By selecting the optimal insertion position for each order, the greedy repair operator significantly improves solution quality. However, due to its sequential insertion approach, it may lack prioritization for nodes that have a substantial impact on total costs.

**Regret repair**: The regret repair operator prioritizes nodes that could cause the highest cost increase if not inserted into their optimal positions. To achieve this, it calculates the differences between the costs of inserting a node into its best and second-best positions. This difference is referred to as the regret value. A higher regret value indicates a more significant cost impact, making it crucial to insert such nodes into their best positions to minimize potential future costs. Compared to the greedy repair operator, this operator improves repair efficiency and solution quality by focusing on nodes with high regret values. However, its tendency to prioritize high-regret nodes may narrow the search space, increasing the risk of becoming trapped in local optima if overused.

### 4.3. Local search

Local search is a class of heuristic algorithms designed for complex optimization problems. Its core idea involves applying local perturbations to the current solution to explore better solutions. After updating the solution, the algorithm performs a local search to further refine and optimize it. In this study, the 2-opt exchange operator is used to search the neighborhood of the current solution, as outlined in Algorithm 4. Specifically, the 2-opt operator swaps any two nodes within the same delivery route until no further improvements can be achieved. This approach optimizes the node arrangement within each delivery route, resulting in a more efficient delivery plan.

**Algorithm 4:** Framework of LocalSearch

**Input:** Parameters: a set of riders $C$, the sets of exchangeable nodes $set_i$ for each node $i$

**foreach** *rider* $c \in C$ **do**
  Initialize $num \leftarrow$ size of $C$, $cost_{best} \leftarrow$ current cost $cost_c$;
  $index_1 \leftarrow -1$, $index_2 \leftarrow -1$;
  **for** $i \leftarrow 0$ **to** $num-1$ **do**
    **for** $j \leftarrow i+1$ **to** $num$ **do**
      **if** $j \in set_i$ **then**
        Swap site $i$ and $j$;
        Update the current cost $cost_c$;
        **if** $cost_c < cost_{best}$ **then**
          Update $cost_{best} \leftarrow cost_c$;
          Update $index_1 \leftarrow i$, $index_2 \leftarrow j$;
        Swap site $i$ and $j$ to restore the route;
  **if** $index_1 \geq 0$ ***and*** $index_2 \geq 0$ **then**
    Apply the best swap at $index_1, index_2$;

It is important to ensure that the order of the restaurant and customer nodes is correctly maintained during the node swaps. Specifically, the rider should visit the restaurant node before reaching the customer node. In Algorithm 4, the set of exchangeable nodes for a given node $i$ is defined as follows: if $i$ is a supplier node, the set includes all nodes preceding its corresponding customer node; if $i$ is a customer node, the set includes all nodes following its corresponding supplier node. A node $j$ is considered exchangeable with $i$ if it belongs to this set.

### 4.4. Feasibility check and objective calculation

A unique characteristic of the food delivery routing problem is that each order includes both a restaurant node and a customer node. This requires a clear encoding scheme to effectively represent and solve the model. To address this, we adopt a natural number encoding method to represent delivery routes. In this scheme, the distribution center is represented by the number 0, while the node set $\{1,2,3,\ldots,2n\}$ represents all restaurant and customer nodes. Odd indices in the node set denote restaurant nodes, and even indices represent customer nodes. For any order $i$, its restaurant and customer nodes are represented as $2i-1$ and $2i$.

Heuristic algorithms often risk becoming trapped in local optima during the search process, particularly for problems with strict constraints. To address this issue, some hard constraints can be relaxed into soft constraints by incorporating penalty terms into the objective function for violations. However, this approach requires handling infeasible solutions and conducting more complex feasibility checks and objective function evaluations. Agra et al. (2013) and Hu et al. (2018) proposed an effective method to evaluate solution feasibility by analyzing the maximum rider load and the latest arrival time for each delivery route. A solution is deemed feasible only if the maximum rider load on every route does not exceed the rider's load capacity limit and the latest arrival time at each node satisfies its service time window.

A delivery solution is defined as $H=\{h_1,h_2,h_3,\ldots,h_{|K|}\}$, consisting of $|K|$ delivery routes, where each route corresponds to a rider and includes all nodes associated with the rider's assigned orders as well as the distribution center. $h_k=\{\varepsilon_1^k,\varepsilon_2^k,\varepsilon_3^k,\ldots,\varepsilon_{|h_k|}^k\}$ represents the delivery route for rider $k$, and each route $h_k$ contains $|h_k|$ nodes, with the $i$th node denoted as $\varepsilon_i^k$. The latest arrival time for rider $k$ at node $i$ is represented as $\tau_k(i,\Gamma_k)$, and $\Gamma_k$ is the uncertainty budget for service time. According to the formulas proposed by Agra et al. (2013) and Hu et al. (2018), the latest arrival time at each node can be expressed as:

$$\tau_k(i,\Gamma_k)=\begin{cases}0, & \text{if } i=1,\\ \max\left(\alpha_{\varepsilon_i^k},\ \tau_k(i-1,\Gamma_k)+t_{\varepsilon_{i-1}^k,\varepsilon_i^k}+\bar{s}_{\varepsilon_{i-1}^k}\right), & \text{if } 2\le i\le|h_k|,\ \Gamma_k=0,\\ \max\Big(\alpha_{\varepsilon_i^k},\ \tau_k(i-1,\Gamma_k-1)+t_{\varepsilon_{i-1}^k,\varepsilon_i^k} & \\ \qquad +\bar{s}_{\varepsilon_{i-1}^k}+\hat{s}_{\varepsilon_{i-1}^k},\Big) & \text{if } i\le|h_k|,\ 1\le\Gamma_k\le i-1,\\ \qquad \tau_k(i-1,\Gamma_k)+t_{\varepsilon_{i-1}^k,\varepsilon_i^k}+\bar{s}_{\varepsilon_{i-1}^k}, & \\ \tau_k(i,\Gamma_k-1), & \text{if } 2\le i\le\Gamma_k.\end{cases} \tag{37}$$

In food delivery operations, the calculation of a rider's maximum load differs from traditional VRPTW due to the dual operations of pickup and delivery. Given the stochastic nature of order demand, the demands at a restaurant node and its corresponding customer

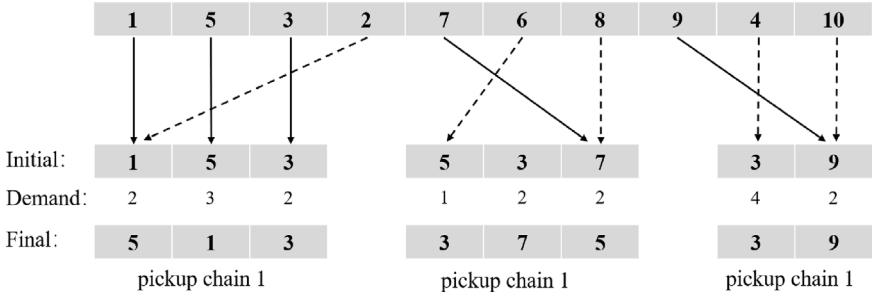



**Fig. 4.** Illustration of "pickup chain" generation.
—— Indicates that the newly arrived restaurant node is inserted into the pick-up chain.
------ Indicates that the restaurant node corresponding to the completed customer node is removed from the delivery chain.

node must remain consistent. However, the relative positions of these nodes within the route may change dynamically as the solution is updated, increasing the complexity of accurately calculating the rider's maximum load.

To simplify calculations while ensuring accuracy, we focus solely on the rider's load at restaurant nodes, excluding customer nodes. This approach is justified because restaurant nodes contribute to the rider's maximum load, while customer nodes reduce it. Consequently, the maximum load on a rider's delivery route will always occur at one of the restaurant nodes. If the maximum load at restaurant nodes does not exceed the rider's capacity, the load at all other restaurant and customer nodes along the route will also satisfy the capacity constraint.

The delivery route $\varepsilon_i^k$ for each rider $k$ is divided into multiple pickup chains, consisting solely of restaurant nodes. Starting from the first restaurant node the rider visits, this node is recorded and inserted into the initial pickup chain. Following the route sequence, each subsequent restaurant node directly visited by the rider is added to the initial pickup chain until the first customer node is encountered, forming the first initial pickup chain. Once the rider delivers to a customer node, the restaurant nodes corresponding to completed deliveries are removed from the chain. As the rider reaches new restaurant nodes, these nodes are appended to the end of the current pickup chain, forming updated pickup chains. This process continues until all nodes on the route have been traversed. Finally, all initial pickup chains are sorted in descending order based on their total demand, resulting in the final pickup chains. The procedure is illustrated in Fig. 4, where odd numbers represent restaurant nodes, even numbers represent customer nodes, and $(2i-1, 2i)$ represent two corresponding nodes of an order.

Each route $h_k = \{\sigma_1^k, \sigma_2^k, \sigma_3^k, \ldots, \sigma_{\delta_k}^k\}$ is divided into $\delta_k$ pickup chains, where $\sigma_m^k = \{\theta_1^{mk}, \theta_2^{mk}, \theta_3^{mk}, \ldots, \theta_{|\sigma_m^k|}^{mk}\}$ represents the $m$th pickup chain for rider $k$, containing $|\sigma_m^k|$ restaurant nodes. The node with the $l$th largest demand in the $m$th pickup chain is denoted as $\theta_l^{mk}$. Since only restaurant node demands are considered, the maximum demand uncertainty budget for each pickup chain is set to half of the route's overall maximum demand uncertainty budget $\Lambda_r$.

The maximum potential load for rider $k$, denoted as $\Pi^k(\Lambda_r)$, is defined as the maximum load $\Pi^{\sigma_m^k}(\Lambda_r/2)$ among all pickup chains along the route. Mathematically, they can be expressed as:

$$\Pi^{\sigma_m^k}(\Lambda_r/2) = \begin{cases} \sum_{l=1}^{|\sigma_m^k|} \bar{r}_{\theta_l^{mk}} + \sum_{l=1}^{\Lambda_r/2} \hat{r}_{\theta_l^{mk}}, & \text{if } \Lambda_r/2 \le |\sigma_m^k| \\ \Pi^{\sigma_m^k}(\Lambda_r/2-1), & \text{if } \Lambda_r/2 > |\sigma_m^k| \end{cases}, \tag{38}$$

$$\Pi^k(\Lambda_r) = \max\left\{\Pi^{\sigma_1^k}(\Lambda_r/2), \Pi^{\sigma_2^k}(\Lambda_r/2), \ldots, \Pi^{\sigma_{\delta_k}^k}(\Lambda_r/2)\right\}. \tag{39}$$

As hard constraints are relaxed into soft constraints, penalties for overtime and overload must be incorporated into the objective function. Let $Z(H)$ denote the objective value of delivery solution $H$ under model $M_1$, as defined by Eq. (24). Then, the total cost of solution $H$ under soft constraints can be expressed as follows:

$$C(H) = Z(H) + f_3 \sum_{k=1}^{|K|} \sum_{i=1}^{|h_k|} \max(0, \tau_k(i, \Gamma_k) - \beta_{\varepsilon_i^k}) + f_4 \sum_{k=1}^{|K|} \max(0, \Pi^k(\Lambda_r) - L). \tag{40}$$

Here, $f_3$ and $f_4$ represent the penalty costs per unit of overtime and overload, respectively.

## 5. Numerical experiments

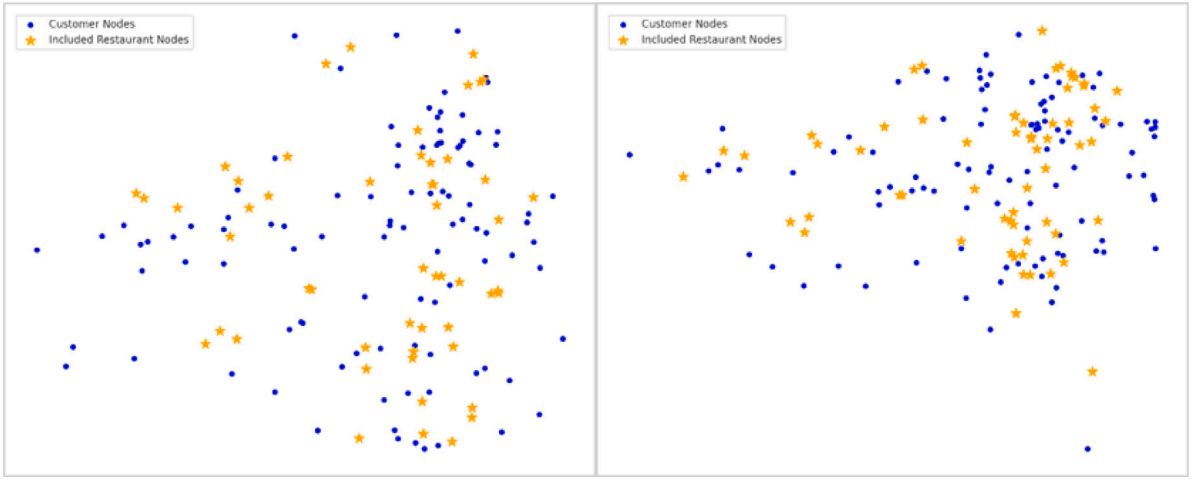
The instances used in this section's experiments are generated based on the Meal Delivery Routing Problem dataset introduced by Reyes et al. (2018). We begin by outlining the design of the computational experiments in detail to assess the solution quality produced by the proposed algorithm. Following this, we conduct a series of experiments and present the results to validate the method's effectiveness in meeting the target service levels.

All experiments were conducted on a workstation equipped with an Intel(R) Xeon(R) Gold 5218R CPU (2.10 GHz) and 32 GB of RAM. The algorithm was implemented in Java (IntelliJ IDEA) and compared against solutions generated by the ILOG CPLEX 12.6.1 solver for performance benchmarking.

### 5.1. Experimental setting

Reyes et al. (2018) introduced a standard dataset for the Meal Delivery Routing Problem, consisting of 10 instance sets. For this study, we modified these instances by randomly selecting 100 orders and 50 riders from each set to create new test cases. Each generated instance was designed to ensure feasibility by confirming that each order could be served by an eligible rider while adhering to both order time windows and rider load capacity constraints. The customer and restaurant coordinates, order-restaurant associations, order times, and service time windows in the generated instances were preserved from the original dataset.

Fig. 5 presents scatter plots illustrating the spatial distribution of customer and restaurant nodes for instances 01o100 and 02o100. The service time for all nodes, including both customers and restaurants, is fixed at four minutes. Riders travel at an average speed of 320 meters per minute. Order demands are generated following a normal distribution with a mean of 3 and a variance of 1, and then rounded to a random integer between 1 and 5. Each rider has a load capacity of 15 units. All riders are assumed to start and end their routes at a distribution center, whose coordinates are calculated as the mean of all restaurant locations. The distances between nodes are computed using Euclidean distance based on their coordinates. Considering that riders typically use electric bicycles in real-world delivery settings, with daily rental costs ranging from 30 to 80 RMB, the activation cost per rider




(a) Instance 01o100 (b) Instance 02o100

**Fig. 5.** Illustrative example: Instances 01o100 and 02o100.

**Table 5**
The parameter combinations.

| No. | $\chi$ | $\xi$ | $\omega$ | $\lambda$ | No. | $\chi$ | $\xi$ | $\omega$ | $\lambda$ |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 0.1 | 0.8 | 0.1 | 0.95 | 6 | 0.3 | 0.99 | 0.1 | 0.98 |
| 2 | 0.1 | 0.9 | 0.3 | 0.98 | 7 | 0.5 | 0.8 | 0.5 | 0.98 |
| 3 | 0.1 | 0.99 | 0.5 | 0.99 | 8 | 0.5 | 0.9 | 0.1 | 0.99 |
| 4 | 0.3 | 0.8 | 0.3 | 0.99 | 9 | 0.5 | 0.99 | 0.3 | 0.95 |
| 5 | 0.3 | 0.9 | 0.5 | 0.95 | – | – | – | – | – |

**Table 6**
Experimental results for each parameter.

| $\chi$ | Obj | $\xi$ | Obj | $\omega$ | Obj | $\lambda$ | Obj |
|---|---|---|---|---|---|---|---|
| 0.1 | 1258 | 0.8 | 1251 | 0.1 | 1253 | 0.95 | 1252 |
| 0.3 | 1249 | 0.9 | 1259 | 0.3 | 1256 | 0.98 | 1257 |
| 0.5 | 1256 | 0.99 | 1252 | 0.5 | 1254 | 0.99 | 1254 |

is standardized at 50 RMB, and the delivery cost is calculated at 0.003 RMB per meter.

To enhance the performance of the QL-LNS algorithm, four key hyperparameters are tuned: the learning rate ($\chi$), discount factor ($\varsigma$), initial acceptance probability ($\omega$), and cooling coefficient ($\lambda$). Each parameter is examined at three levels: $\chi \in \{0.1, 0.3, 0.5\}$, $\varsigma \in \{0.8, 0.9, 0.99\}$, $\omega \in \{0.1, 0.3, 0.5\}$, and $\lambda \in \{0.95, 0.98, 0.99\}$. An L9 orthogonal array (Table 5) is utilized to systematically explore parameter combinations (Jolfaei & Alinaghian, 2024). Experiments are performed on a medium-sized instance (01o60); each configuration is run 30 times, and mean results are used for evaluation. As summarized in Table 6, the best-performing settings are $\chi = 0.3$, $\varsigma = 0.8$, $\omega = 0.3$, and $\lambda = 0.98$.

The parameter values for the uncertainty sets, $U_t$ and $U_r$, are determined based on the recommendations provided by Hu et al. (2018). Specifically, the nominal values are derived from the service times and demands associated with each instance, with a maximum deviation of 20% from these values. Furthermore, the uncertainty budget parameters are set at 0.5 to ensure consistency with the proposed framework.

### 5.2. Effectiveness evaluation of the Q-learning strategy

To evaluate the effectiveness of the Q-learning strategy in improving the performance of the LNS algorithm for problem solving, a series of comparative experiments was designed and conducted. The traditional LNS algorithm served as the baseline, and the solution quality and stability of both algorithms were compared across multiple test instances to assess the improvements introduced by Q-learning. The experimental results are summarized in Table 7. The results demonstrate that the QL-LNS consistently outperformed the traditional LNS in most test scenarios, achieving better objective values and exhibiting stronger convergence capability. The most significant improvement was observed in instance 02o100, with a performance gain of 6.4%, and the overall average improvement rate reached 2.5%. The relatively lower overall improvement is attributed to small-scale instances where both QL-LNS and LNS achieved optimal solutions. These findings confirm the positive effect of the Q-learning strategy in enhancing neighborhood search efficiency and guiding the search process toward high-quality solution spaces.

### 5.3. Comparison between QL-LNS and SARSA-based LNS

To further evaluate the performance of the proposed QL-LNS algorithm in solving the meal delivery routing optimization problem, this study selects the SARSA (State–Action–Reward–State–Action) algorithm, a representative method in reinforcement learning, as a benchmark for comparison. Both SARSA and Q-learning are temporal-difference methods; however, SARSA updates its value estimates based on the actions dictated by the current policy, rather than greedy actions. This characteristic can result in greater policy stability under certain conditions. By comparing the performance of the two algorithms under identical meal delivery task settings, a more objective evaluation of the overall advantages of QL-LNS can be obtained. The comparative experiments were conducted under identical delivery scenarios, initial parameters, and environment configurations. These experimental results are presented in Table 8.

The experimental data presented in Table 8 clearly demonstrate that the QL-LNS algorithm outperforms SARSA in most test instances by achieving better average objective function values and significantly shorter solution times in the majority of scenarios. Except for instances 01o60 and 02o40, the solutions obtained by QL-LNS are consistently superior in quality, exhibiting stronger stability and convergence capabilities. As the problem scale increases, QL-LNS shows even more pronounced advantages, particularly in medium- and large-scale scenarios, where its superiority in both solution quality and computational efficiency becomes increasingly evident. For instance, in instance 02o80, QL-LNS outperforms SARSA by 2.8%, and in instance 03o80, the improvement reaches 3.3%, further validating the robustness and effectiveness of QL-LNS in handling complex problems. On average, QL-LNS achieves a 0.6% improvement in solution quality compared to SARSA. Although this improvement may not appear statistically significant, it

**Table 7**
Experimental results of QL-LNS and LNS.

| Instance group | N.O. | QL-LNS | | | LNS | | | GAP |
|---|---|---|---|---|---|---|---|---|
| | | Obj_best | Obj_avg | Time | Obj_best | Obj_avg | Time | |
| 1 | 20 | 551 | 553 | 2.76 | 551 | 554 | 2.66 | 0.0% |
| | 40 | 914 | 935 | 16.16 | 946 | 954 | 25.44 | 3.5% |
| | 60 | 1247 | 1263 | 78.55 | 1264 | 1288 | 69.82 | 1.4% |
| | 80 | 1580 | 1616 | 198.39 | 1644 | 1678 | 195.07 | 4.0% |
| | 100 | 1914 | 1988 | 327.15 | 2002 | 2068 | 261.53 | 4.6% |
| 2 | 20 | 524 | 525 | 2.21 | 524 | 525 | 1.19 | 0.0% |
| | 40 | 814 | 825 | 15.30 | 820 | 835 | 16.12 | 0.7% |
| | 60 | 1068 | 1086 | 62.35 | 1084 | 1103 | 84.32 | 1.5% |
| | 80 | 1421 | 1475 | 163.36 | 1482 | 1506 | 236.80 | 4.3% |
| | 100 | 1782 | 1811 | 385.78 | 1897 | 1993 | 344.44 | 6.4% |
| 3 | 20 | 603 | 603 | 2.31 | 603 | 603 | 1.35 | 0.0% |
| | 40 | 861 | 868 | 15.45 | 867 | 873 | 18.31 | 0.8% |
| | 60 | 1257 | 1268 | 105.27 | 1309 | 1324 | 97.34 | 4.1% |
| | 80 | 1436 | 1458 | 347.06 | 1494 | 1512 | 445.34 | 4.0% |
| | 100 | 1847 | 1882 | 497.31 | 1882 | 1895 | 595.96 | 1.9% |
| Avg | – | – | – | – | – | – | – | 2.5% |

**Notes :** The "N.O." column in the table indicates the total number of orders in the instance.

**Table 8**
Experimental results of QL-LNS and SARSA.

| Instance group | N.O. | QL-LNS | | | | SARSA | | | | GAP |
|---|---|---|---|---|---|---|---|---|---|---|
| | | N.R. | Obj_best | Obj_avg | Time | N.R. | Obj_best | Obj_avg | Time | |
| 1 | 20 | 5 | 551 | 553 | 2.76 | 5 | 551 | 553 | 2.13 | 0.0% |
| | 40 | 7 | 914 | 935 | 16.16 | 7 | 914 | 943 | 38.49 | 0.0% |
| | 60 | 9 | 1247 | 1263 | 78.55 | 9 | 1243 | 1271 | 80.28 | −0.3% |
| | 80 | 11 | 1580 | 1616 | 198.39 | 11 | 1598 | 1648 | 235.80 | 1.1% |
| | 100 | 14 | 1914 | 1988 | 327.15 | 14 | 1936 | 2010 | 539.31 | 1.1% |
| 2 | 20 | 5 | 524 | 525 | 2.21 | 5 | 524 | 525 | 6.52 | 0.0% |
| | 40 | 6 | 814 | 825 | 15.30 | 6 | 813 | 825 | 15.04 | −0.1% |
| | 60 | 8 | 1068 | 1086 | 62.35 | 8 | 1072 | 1096 | 136.02 | 0.4% |
| | 80 | 11 | 1421 | 1475 | 163.36 | 11 | 1461 | 1483 | 291.27 | 2.8% |
| | 100 | 13 | 1782 | 1811 | 385.78 | 13 | 1785 | 1817 | 466.42 | 0.1% |
| 3 | 20 | 6 | 603 | 603 | 2.31 | 6 | 603 | 603 | 2.92 | 0.0% |
| | 40 | 7 | 861 | 868 | 15.45 | 7 | 861 | 867 | 17.67 | 0.0% |
| | 60 | 10 | 1257 | 1268 | 105.27 | 10 | 1259 | 1271 | 275.54 | 0.2% |
| | 80 | 11 | 1436 | 1458 | 347.06 | 12 | 1483 | 1502 | 392.63 | 3.3% |
| | 100 | 13 | 1847 | 1882 | 497.31 | 13 | 1849 | 1864 | 783.15 | 0.1% |
| Avg | – | – | – | – | – | – | – | – | – | 0.6% |

**Notes :** The "N.R." column in the table indicates the number of riders utilized in the solution.

is largely due to the fact that in smaller instances (e.g., those with 20 or 40 orders), both algorithms can easily find comparable solutions, thus diluting the overall average gain. Nevertheless, this also suggests that QL-LNS is capable of maintaining solution quality comparable to SARSA for small-scale problems while demonstrating greater potential in large-scale scenarios.

In terms of computational time, QL-LNS also shows a clear advantage. In most test cases, QL-LNS requires significantly less time to achieve equal or better solutions than SARSA, especially in medium- and large-scale tasks. For example, in instance 03o100, both algorithms found similar optimal solutions, but SARSA required 783.15 s to solve the instance, compared to just 497.31 s for QL-LNS. This indicates that SARSA incurs a higher exploration cost when faced with complex decision paths, while QL-LNS benefits from the synergy between its large neighborhood search strategy and reinforcement learning, thereby improving search efficiency.

In summary, QL-LNS not only delivers stable and reliable performance in small-scale problems but also maintains high solution quality and computational efficiency in large-scale and computationally intensive delivery routing scenarios. This demonstrates the algorithm's strong practicality and scalability. The systematic comparison with SARSA further confirms the comprehensive performance advantages of QL-LNS in solving meal delivery routing optimization problems.

### 5.4. Comparison with CPLEX and ALNS for the deterministic problem

This section evaluates the performance of the proposed QL-LNS algorithm when applied to the deterministic problem (i.e., $M_0$). We compare its solution quality and computational efficiency against the commercial solver CPLEX on small-scale instances. Since CPLEX becomes intractable for larger problems, we then compare QL-LNS with a state-of-the-art ALNS algorithm, replicated based on Wang et al. (2024), for large-scale instances.

The experiments utilized instance set 01, selecting nine small-scale cases with order quantities ranging from two to ten, where the number of riders was set to half the number of orders. The results are presented in Table 9. The findings indicate that both algorithms perform well in small-scale instances, with the objective values obtained by the QL-LNS and ALNS algorithms matching those solved by CPLEX. However, as the number of orders increases, the solution time for CPLEX increases significantly, particularly for instances 01o09 and 01o10, where the solving time exceeds 7200 s. For instances with more than 10 orders, CPLEX is unable to obtain the optimal solution within a reasonable time frame. In contrast, both the QL-LNS and ALNS algorithms demonstrate higher stability and efficiency, with solution times consistently remaining below two seconds.

**Table 9**
Experimental results for small arithmetic cases of CPLEX, QL-LNS, and ALNS algorithms.

| Instance | N.O. | CPLEX | | | QL-LNS | | | | ALNS | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | N.R. | Obj | Time | N.R. | Obj | Time | GAP | N.R. | Obj | Time | GAP |
| 01o02 | 2 | 1 | 90 | 0.06 | 1 | 90 | 0.19 | 0.0% | 1 | 90 | 1.02 | 0.0% |
| 01o03 | 3 | 1 | 99 | 0.16 | 1 | 99 | 0.15 | 0.0% | 1 | 99 | 1.06 | 0.0% |
| 01o04 | 4 | 1 | 112 | 0.25 | 1 | 112 | 0.26 | 0.0% | 1 | 112 | 1.13 | 0.0% |
| 01o05 | 5 | 1 | 117 | 2.23 | 1 | 117 | 0.15 | 0.0% | 1 | 117 | 1.33 | 0.0% |
| 01o06 | 6 | 2 | 186 | 89.03 | 2 | 186 | 0.26 | 0.0% | 2 | 186 | 1.17 | 0.0% |
| 01o07 | 7 | 2 | 202 | 498.30 | 2 | 202 | 0.35 | 0.0% | 2 | 202 | 1.32 | 0.0% |
| 01o08 | 8 | 2 | 206 | 140.53 | 2 | 206 | 0.36 | 0.0% | 2 | 206 | 1.81 | 0.0% |
| 01o09 | 9 | 2 | 215 | 7217.30 | 2 | 215 | 0.39 | 0.0% | 2 | 215 | 1.67 | 0.0% |
| 01o10 | 10 | 2 | 237 | 7206.78 | 2 | 237 | 0.63 | 0.0% | 2 | 237 | 1.87 | 0.0% |
| Avg | – | – | – | – | – | – | – | 0.0% | – | – | – | 0.0% |

As CPLEX fails to obtain optimal solutions within a reasonable time frame for large-scale instances, it is unsuitable as a benchmark for evaluating the solution quality of our algorithm. To further validate the effectiveness of the proposed approach, we conducted experiments on large-scale instances, comparing the performance of the QL-LNS and ALNS algorithms. Instances 01, 02, and 03 were used, generating 15 large-scale cases with order quantities of 20, 40, 60, 80, and 100. Each instance was run 10 times, with the experimental results summarized in Table 10.

The experimental results show that the difference between the optimal and average solutions obtained by the QL-LNS and ALNS algorithms over 10 runs remains within 5%. In small-scale instances with only 20 orders, both QL-LNS and ALNS perform exceptionally well, consistently finding near-optimal solutions with an average gap of less than 0.5%. Notably, in instance 03o100, both algorithms successfully identified the same best solution. As the number of orders increases, this difference slightly rises. For instance, in the larger instance 01o100 with 100 orders, the gap of the QL-LNS algorithm remains at a manageable 3.9%, which is still within an acceptable range.

In contrast, the QL-LNS algorithm outperforms the ALNS algorithm regarding solution quality for large-scale instances. Except for instances 01o20, 02o20, and 03o20 (where both algorithms yield the same optimal solution), the ALNS algorithm consistently produces better solutions in the remaining instances, with an average difference of 1.8%. Fig. 6 illustrates the performance comparison in terms of solution quality and computation time between QL-LNS and ALNS. The blue curve represents the difference in solution quality, while the red curve shows the comparison of computation speed. The performance gap in solution quality widens as the problem size increases. For instance, in instance 02o100, QL-LNS outperforms ALNS by 5.3%. Regarding runtime, we define the Speed up Factor as the ratio of ALNS's runtime to QL-LNS's runtime. A value greater than 1 indicates that QL-LNS runs faster. As shown in the figure, the red curve consistently stays above 1, confirming that QL-LNS maintains a lower average computation time across all instances, thereby demonstrating superior computational efficiency.

### 5.5. The effect of demand and travel time uncertainty

A key feature of this study is the incorporation of robust optimization to account for uncertainties in service time and order demand in food delivery logistics. To assess the robustness of the delivery plans generated by the proposed algorithm, we utilized Monte Carlo simulations to test performance under random scenarios, comparing the robustness of the robust QL-LNS algorithm against the deterministic QL-LNS algorithm in handling randomness. Specifically, we assumed that each order's demand follows a normal distribution with a mean equal to the instance's demand $r$ and a variance of $0.2r$. Similarly, the service time is modeled as a normal distribution with a mean $t_s$ and variance $0.2t_s$ based on the instance's service time. A total of 1000 random scenarios were simulated, evaluating the service level of each delivery plan in each scenario. A delivery plan was considered successful in a given scenario if all orders were completed within their service time windows and each rider's load remained within the set capacity.

To quantify the effectiveness of the plans, we defined three metrics: L0 represents the average probability of satisfying all orders, L1 represents the average probability of meeting all but one order, and L2 represents the average probability of meeting all but two orders. These metrics comprehensively assess the robustness of the delivery plans across various random scenarios.

We conducted experiments on 10 instances, with results presented in Table 11. The "Type" column indicates whether the robust algorithm or the deterministic algorithm was used, "N.R." represents the number of riders required for the delivery plan, and "Obj" denotes the operational cost of the plan. The "Total" column reflects the results when both order demand and service time are uncertain. The results demonstrate that, across all instances, the proposed robust algorithm consistently outperforms the deterministic algorithm. The service level probabilities for deterministic and robust algorithms where demand and service time are stochastic are illustrated in Fig. 7.

When conducting simulation experiments with delivery plans derived from the deterministic algorithm, customer service levels were notably low. When both random variables vary, the probability of serving all customers consistently falls below 10%, reaching a maximum of only 10.9% in instance 01o100. Additionally, the probability of serving all but one customer is 35.0%, while the probability of serving all but two customers peaks at only 60.0%. In contrast, the robust algorithm demonstrated significantly better performance. Except for instance 08o100, the average probability of serving all customers under the deterministic solution in the other instances is 86.92%. Although the lowest probability of serving all customers in instance 08o100 is 32.6%, the probability of serving all but one customer was as high as 82.2%, and the probability of serving all but two customers reached 98.6%. As shown in Fig. 8, the three lines representing the robust results are farther from the center of the circle compared to the curves of the deterministic algorithms, indicating that the robust algorithms maintain higher service levels across all instances.

In Table 8, L0Rob in instance 08o100 is positioned slightly closer to the center, but in the remaining instances, the probability of serving all customers across instances exceeded 79%, with three instances reaching probabilities above 90%. Even in instance 08o100, the robust algorithm clearly outperforms the deterministic algorithm. The performance of L1Rob and L2Rob are even more impressive, with both approaches being close to the outermost circle. The average probability of serving all but one customer generally exceeded 90%, with six instances surpassing 98%, and across all instances, the probability of serving all but two customers reached 99% or higher.

Additionally, we compared the performance of the two algorithms under scenarios with a single stochastic variable. The results are presented in Table 12, where the "Demand" column reflects cases where only order demand is uncertain and service time is deterministic, while

**Table 10**
Comparison of experimental results between QL-LNS and ALNS algorithms.

| Instance group | N.O. | QL-LNS | | | | | ALNS | | | | | GAP |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | N.R. | Obj_best | Obj_avg | Time | Gap | N.R. | Obj_best | Obj_avg | Time | Gap | |
| 1 | 20 | 5 | 551 | 553 | 2.76 | 0.4% | 5 | 551 | 553 | 2.84 | 0.4% | 0.0% |
| | 40 | 7 | 914 | 935 | 16.16 | 2.3% | 7 | 935 | 951 | 30.84 | 1.6% | 2.4% |
| | 60 | 9 | 1247 | 1263 | 78.55 | 1.3% | 9 | 1259 | 1273 | 154.73 | 1.1% | 1.0% |
| | 80 | 11 | 1580 | 1616 | 198.39 | 2.3% | 12 | 1628 | 1652 | 279.25 | 1.5% | 3.0% |
| | 100 | 14 | 1914 | 1988 | 327.15 | 3.9% | 15 | 1995 | 2059 | 579.29 | 3.2% | 4.2% |
| 2 | 20 | 5 | 524 | 525 | 2.21 | 0.2% | 5 | 524 | 525 | 2.14 | 0.2% | 0.0% |
| | 40 | 6 | 814 | 825 | 15.30 | 1.3% | 6 | 818 | 827 | 17.68 | 1.1% | 0.4% |
| | 60 | 8 | 1068 | 1086 | 62.35 | 1.6% | 8 | 1079 | 1095 | 76.38 | 1.5% | 1.0% |
| | 80 | 11 | 1421 | 1475 | 163.36 | 3.8% | 11 | 1471 | 1490 | 318.53 | 1.3% | 3.5% |
| | 100 | 13 | 1782 | 1811 | 385.78 | 1.6% | 14 | 1876 | 1922 | 745.42 | 2.4% | 5.3% |
| 3 | 20 | 6 | 603 | 603 | 2.31 | 0.0% | 6 | 603 | 603 | 2.39 | 0.0% | 0.0% |
| | 40 | 7 | 861 | 868 | 15.45 | 0.9% | 7 | 862 | 868 | 30.78 | 0.6% | 0.2% |
| | 60 | 10 | 1257 | 1268 | 105.27 | 0.8% | 10 | 1277 | 1308 | 143.25 | 2.5% | 1.5% |
| | 80 | 11 | 1436 | 1458 | 347.06 | 1.6% | 11 | 1471 | 1493 | 495.35 | 1.5% | 2.4% |
| | 100 | 13 | 1847 | 1882 | 497.31 | 1.9% | 14 | 1877 | 1943 | 1137.98 | 3.5% | 1.7% |
| Avg | – | – | – | – | – | – | – | – | – | – | – | 1.8% |

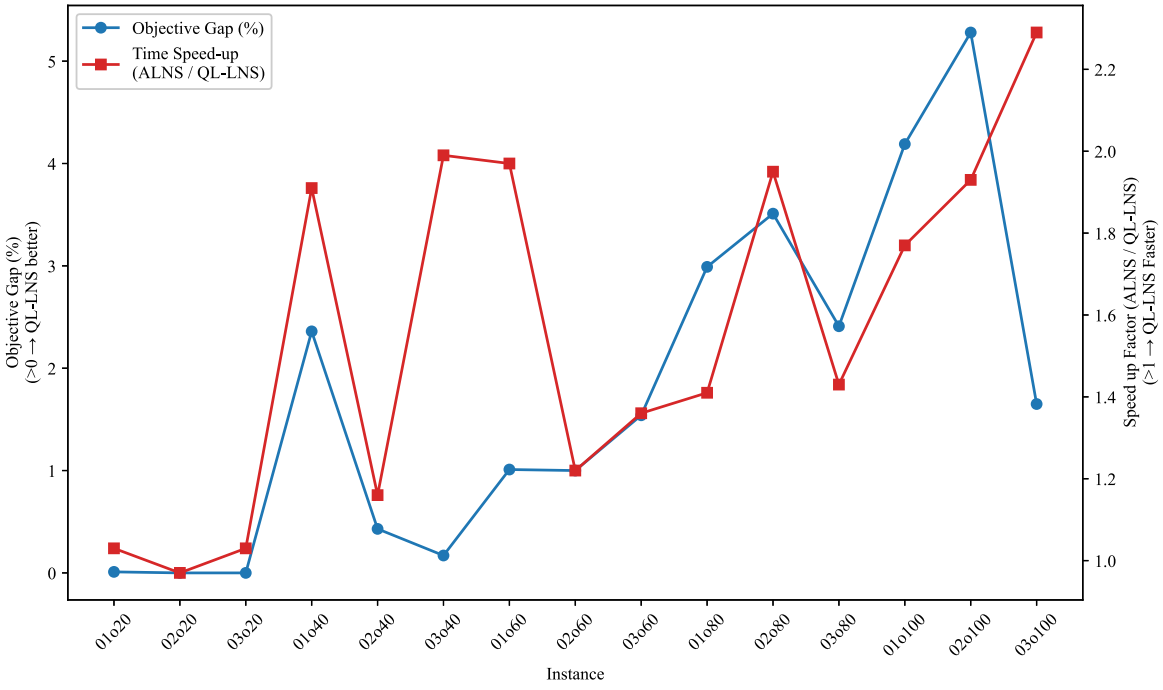



**Fig. 6.** QL-LNS vs. ALNS: Quality & speed in one view.

the "Service Time" column represents cases where only service time is uncertain and order demand is deterministic. The results indicate that, regardless of which stochastic variable is considered, the robust algorithm consistently achieves higher service levels than the deterministic algorithm. The difference lies primarily in the magnitude of the improvement in service level. Furthermore, service level probability radar charts were plotted for scenarios where demand and service time were independently stochastic, showing the probabilities of serving all customers, serving all but one customer, and serving all but two customers, as illustrated in Fig. 8.

As shown clearly in Table 12, regardless of the type of uncertainty scenario, the robust algorithm consistently outperforms the deterministic algorithm in solution quality across all instances. Except for instance 08o100, the robust solutions in all other cases achieve service levels above 80% in various scenarios and are capable of serving all customers. However, the performance of the deterministic algorithm varies significantly across different uncertainty scenarios. Specifically, in scenarios with uncertain service times, the service level of the deterministic solution is relatively close to that in the scenario where both demand and service time are stochastic, as illustrated in Fig. 8(b). In the figure, the three curves corresponding to the robust algorithm are relatively concentrated and close to the outermost circle, indicating high service capability under various conditions. In contrast, the curves of the deterministic algorithm gradually expand outward from a position near the center, yet never surpass the performance of the robust algorithm.

By comparison, the service level of the deterministic solution is significantly higher in the scenario with uncertain demand than in the one with uncertain service time. For example, among all instances, the probability of serving all customers using the deterministic solution in the uncertain demand scenario is lowest in instance 06o100 at 27.5%, whereas this probability drops sharply to 5.8% in the uncertain service time scenario—a reduction of more than fivefold. Meanwhile, in instances 07o100, 08o100, 09o100, and 10o100, the probability of serving all customers under the uncertain demand scenario exceeds 90% for the deterministic solutions. Except for instance 06o100, the

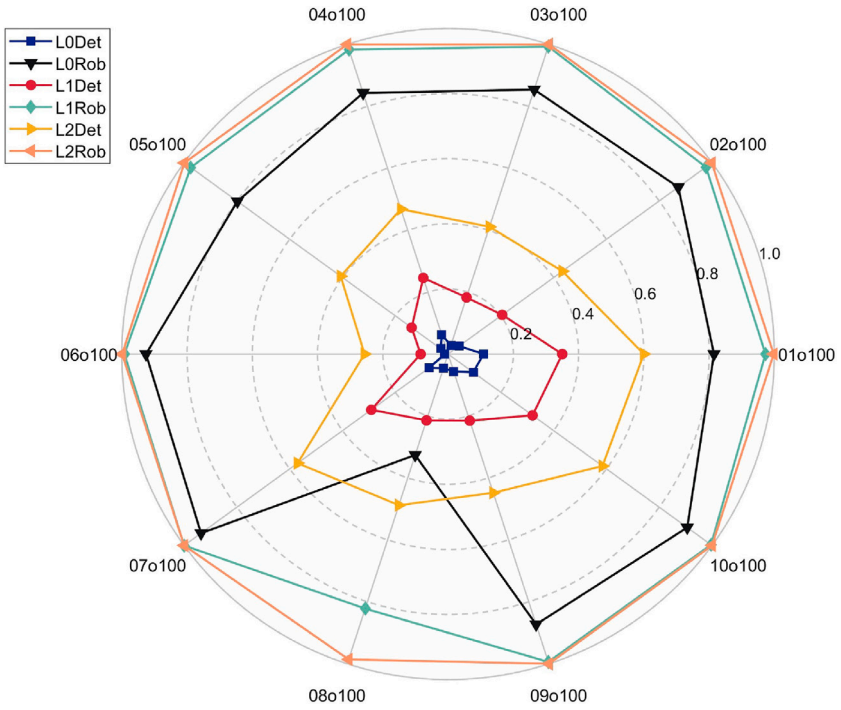
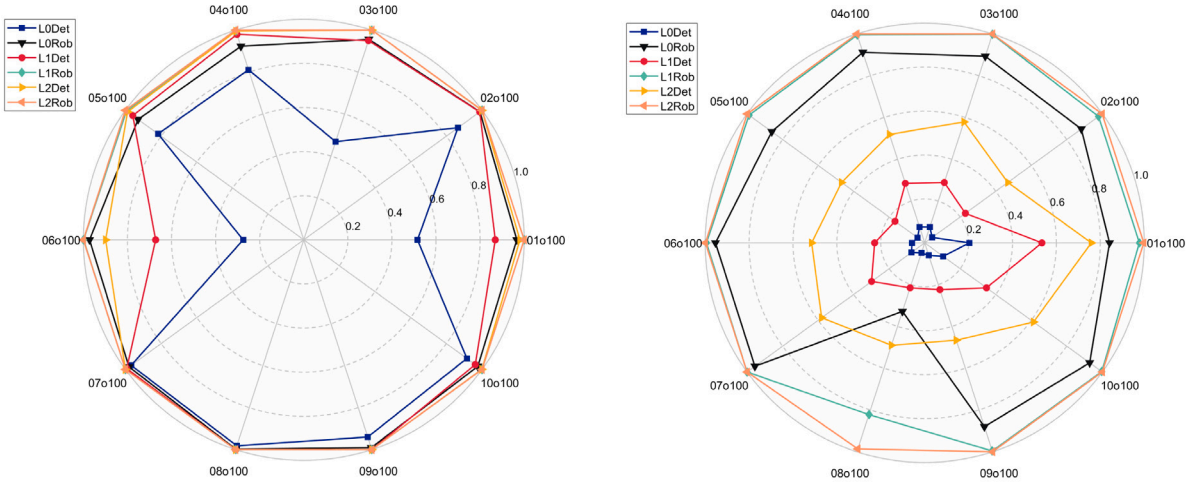



**Fig. 7.** Service level probabilities for deterministic and robust algorithms.




(a) Stochastic demand

(b) Stochastic service time

**Fig. 8.** Service level probabilities for scenarios where demand and service time are independently stochastic.

probability of failing to serve at most two customers in the uncertain demand scenario approaches 100% for the other instances; even for the worst-performing instance 06o100, this probability still reaches 89.9%. As shown clearly in Fig. 8(a), the three curves representing the deterministic solution—L0Det, L1Det, and L2Det—are farther from the center than in Table 8, and both L1Det and L2Det are close to the outer ring, indicating a notable improvement in service level.

These findings highlight the relatively minimal impact of demand variability on service levels compared to the effects of service time variability. Overall, service levels appeared to be more sensitive to fluctuations in service time. This phenomenon can be attributed to the assignment of fewer orders to riders to prevent violations of time window constraints, thus reducing the risk of overload due to customer demand uncertainty. Consequently, in scenarios where only demand uncertainty exists, the deterministic algorithm can still maintain a high level of service performance.

### 5.6. Sensitivity analysis

#### 5.6.1. Demand analysis

The previous experiments indicate that random fluctuations in order demand have a limited impact on the delivery plan. To further examine the influence of order demand on the robustness of delivery plans, we conducted additional experiments using instances 01o100 and 02o100. These experiments focused on scenarios where only order demand variability was simulated through Monte Carlo simulations. Given that customer demand uncertainty is relatively small compared to rider load capacity, he rider load capacity was proportionally reduced to

**Table 11**
Comparison of experimental results between robust QL-LNS and deterministic QL-LNS under two random variables.

| Instance | Type | N.R. | Obj | Increase | Total | | |
|---|---|---|---|---|---|---|---|
| | | | | | L0 | L1 | L2 |
| 01o100 | Det | 14 | 1914 | – | 10.9% | 35.0% | 60.0% |
| | Rob | 15 | 2031 | 6.1% | 81.5% | 97.3% | 99.8% |
| 02o100 | Det | 13 | 1782 | – | 4.2% | 20.5% | 43.3% |
| | Rob | 13 | 1855 | 4.1% | 87.2% | 97.7% | 99.9% |
| 03o100 | Det | 13 | 1847 | – | 2.8% | 18.3% | 41.1% |
| | Rob | 14 | 1891 | 2.4% | 85.4% | 99.3% | 100.0% |
| 04o100 | Det | 13 | 1678 | – | 6.2% | 24.6% | 46.8% |
| | Rob | 13 | 1697 | 1.1% | 84.3% | 98.3% | 100.0% |
| 05o100 | Det | 12 | 1675 | – | 2.8% | 13.8% | 40.7% |
| | Rob | 13 | 1729 | 3.2% | 79.8% | 97.5% | 99.9% |
| 06o100 | Det | 12 | 1941 | – | 1.0% | 8.4% | 25.6% |
| | Rob | 12 | 1985 | 2.3% | 92.6% | 99.2% | 99.9% |
| 07o100 | Det | 12 | 1844 | – | 7.2% | 29.1% | 57.0% |
| | Rob | 12 | 1910 | 3.6% | 93.6% | 100.0% | 100.0% |
| 08o100 | Det | 14 | 2054 | – | 4.6% | 21.4% | 48.8% |
| | Rob | 13 | 2118 | 3.1% | 32.6% | 82.2% | 98.6% |
| 09o100 | Det | 12 | 2263 | – | 5.6% | 21.5% | 44.8% |
| | Rob | 13 | 2349 | 3.8% | 87.3% | 99.4% | 100.0% |
| 10o100 | Det | 13 | 2591 | – | 9.5% | 32.0% | 58.5% |
| | Rob | 14 | 2619 | 1.1% | 90.6% | 99.6% | 100.0% |

**Table 12**
Comparison of experimental results between robust QL-LNS and deterministic QL-LNS under separate random variables.

| Instance | Type | Demand | | | Service time | | |
|---|---|---|---|---|---|---|---|
| | | L0 | L1 | L2 | L0 | L1 | L2 |
| 01o100 | Det | 51.6% | 86.9% | 98.1% | 20.4% | 53.4% | 75.9% |
| | Rob | 96.7% | 100.0% | 100.0% | 84.2% | 97.8% | 99.8% |
| 02o100 | Det | 86.5% | 98.7% | 99.9% | 4.2% | 22.9% | 46.8% |
| | Rob | 98.8% | 100.0% | 100.0% | 88.2% | 97.9% | 99.9% |
| 03o100 | Det | 46.8% | 95.0% | 100.0% | 7.6% | 28.9% | 57.9% |
| | Rob | 95.6% | 100.0% | 100.0% | 89.3% | 99.8% | 100.0% |
| 04o100 | Det | 81.1% | 98.0% | 99.5% | 7.6% | 28.5% | 51.9% |
| | Rob | 92.3% | 99.6% | 100.0% | 91.2% | 99.5% | 100.0% |
| 05o100 | Det | 81.7% | 95.9% | 98.8% | 4.0% | 16.7% | 46.8% |
| | Rob | 92.9% | 99.4% | 100.0% | 86.1% | 98.9% | 99.9% |
| 06o100 | Det | 27.5% | 67.2% | 89.9% | 5.8% | 22.8% | 51.5% |
| | Rob | 97.2% | 99.9% | 100.0% | 95.3% | 99.4% | 99.9% |
| 07o100 | Det | 96.8% | 99.3% | 100.0% | 7.4% | 29.9% | 58.0% |
| | Rob | 98.0% | 100.0% | 100.0% | 95.6% | 100.0% | 100.0% |
| 08o100 | Det | 98.2% | 100.0% | 100.0% | 4.8% | 21.5% | 49.0% |
| | Rob | 99.8% | 100.0% | 100.0% | 32.8% | 82.2% | 98.6% |
| 09o100 | Det | 94.0% | 100.0% | 100.0% | 6.0% | 22.4% | 46.6% |
| | Rob | 99.1% | 100.0% | 100.0% | 88.0% | 99.6% | 100.0% |
| 10o100 | Det | 91.5% | 96.2% | 99.9% | 10.4% | 34.8% | 61.3% |
| | Rob | 97.6% | 100.0% | 100.0% | 92.9% | 99.7% | 100.0% |

90%, 80%, 70%, and 60% of its original value. This adjustment allows demand fluctuations to constitute a larger proportion of the total load capacity, thereby making the algorithmic effects more pronounced. We applied both algorithms to these adjusted instances, and the comparative results are presented in Tables 13 and 14.

The experimental results indicate that when the rider load capacity is low and fluctuations in order demand are significant, the delivery plans generated by the robust algorithm exhibit higher robustness compared to those obtained from the deterministic algorithm. In this experiment, the range of order demand fluctuations was kept consistent. As rider load capacity decreases, the influence of random factors progressively increases, resulting in a decline in overall service levels. The probability trends for service level in instances 01o100 and 02o100 are illustrated in Fig. 9. Although both algorithms show a gradual decrease in service levels as load capacity is reduced, the decline is more obvious for the deterministic setting. It is observed that the robust algorithm consistently outperforms the deterministic one in terms of having a higher service level (L0, L1, L2) for the same or slightly increased number of riders, as the capacity of each individual rider decreases. This demonstrates the robust algorithm's enhanced capability to account for demand fluctuations and ensure higher levels of service reliability under constrained conditions.

In instance 01o100, the robust algorithm shows a significant advantage over the deterministic algorithm, particularly at lower capacities. For example, at a capacity of 0.6Q, the robust algorithm maintains a probability of approximately 20% for L0 (serving all customers), while the deterministic algorithm drops to 0%. Similarly, for L1 and L2, the robust algorithm achieves probabilities of around 50% and 77%, respectively, whereas the deterministic algorithm fails to maintain any substantial probability at these service levels. This highlights the robust algorithm's superior adaptability in scenarios where demand fluctuations are significant but not excessively challenging. Instance 02o100, on the other hand, presents a more challenging scenario for both algorithms. The probabilities for all service levels are generally lower compared to instance 01o100, and the rate of decline in probabilities is more pronounced as the rider load capacity decreases. Despite this, the robust algorithm still outperforms the deterministic algorithm, maintaining higher service levels under conditions of substantial demand fluctuations.

As depicted in Fig. 9, the robust algorithm consistently generates solutions that maintain higher service levels under significant demand variability, further demonstrating its advantages over the deterministic algorithm. However, it should be noted that while the robust delivery plan enables the system to serve more customers than the deterministic plan, its overall performance still declines compared to earlier service levels. This decline is primarily due to the low rider load capacity, which restricts the ability to accommodate higher levels of random demand. Increasing the number of riders to enhance the residual capacity for handling demand fluctuations could improve service levels but would likely result in higher costs.

### 5.6.2. Sensitivity analysis of uncertainty parameters

In the robust formulation described above, the uncertainty interval and uncertainty budget are key parameters that significantly influence the effectiveness and efficiency of the robust solution. To evaluate their impact on the final delivery plans, we conducted sensitivity analyses on these two parameters independently. The uncertainty budget defines the upper limit on the proportion of orders with uncertainty, while the uncertainty interval represents the range of fluctuations in the random variable. A broader uncertainty interval increases the variability of the random variables, while a narrower interval reduces this variability.

Since these two parameters are independent, they were adjusted separately by holding one constant while varying the other. The analysis first focused on the uncertainty budget, which was set to values of 0, 0.2, 0.4, 0.6, 0.8, and 1.0. An uncertainty budget of 0 signifies that all orders are deterministic, whereas a value of 1.0 indicates that all orders are subject to uncertainty. For this analysis, the uncertainty interval was fixed at 0.2.

Experiments were performed on instances 01o100 and 02o100, with results shown in Table 15. The findings reveal that as the uncertainty budget increases, service levels improve correspondingly. For example, in instance 01o100, when the uncertainty budget is 0 (i.e., no uncertain orders), the average probability of meeting all customer demands is 10.9%. When the uncertainty budget increases to 1.0 (i.e., all orders are uncertain), this probability rises to 94.5%, while the probability of meeting all but two customers reaches 100%. However, the improvement in service levels is accompanied by higher delivery costs and an increase in the number of riders required.

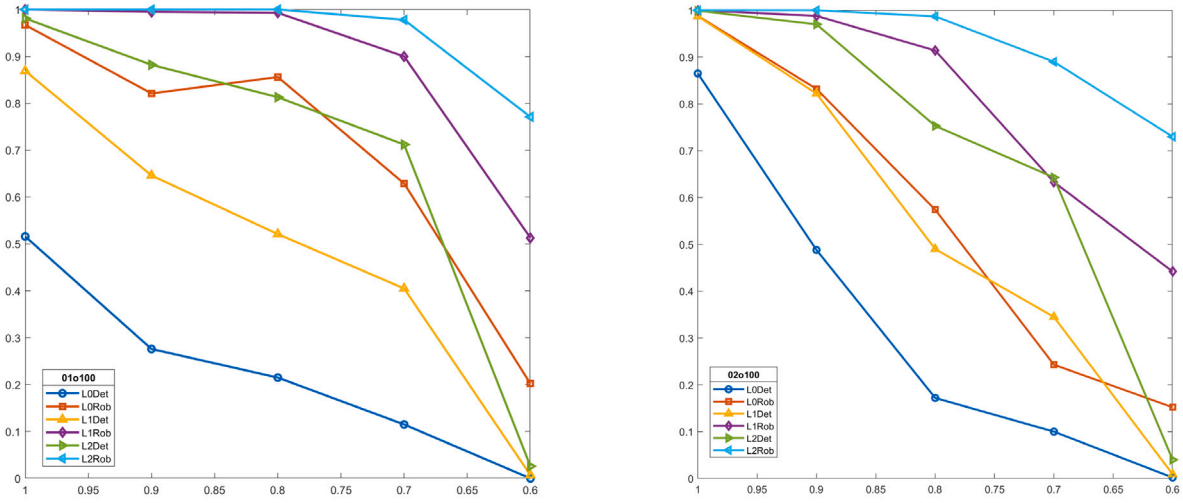
For instance, in instance 01o100, as the uncertainty budget rises from 0.2 to 0.4, the number of riders increases from 14 to 15. This adjustment allows the delivery plan to better accommodate uncertain demands by assigning certain orders to the additional rider, thereby ensuring adherence to time window and load capacity constraints. A

**Table 13**
Experimental results for calculation 01o100 with different rider loading limits.

| Load | Det | | | | | Rob | | | | | Increase | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| limit | N.R. | Obj | L0 | L1 | L2 | N.R. | Obj | L0 | L1 | L2 | N.R. | Obj |
| 1.0Q | 14 | 1914 | 51.6% | 86.9% | 98.1% | 15 | 2031 | 96.7% | 100.0% | 100.0% | 1 | 6.1% |
| 0.9Q | 14 | 1948 | 27.6% | 64.6% | 88.2% | 15 | 2042 | 82.1% | 99.5% | 99.5% | 1 | 4.8% |
| 0.8Q | 14 | 1968 | 21.5% | 52.1% | 81.3% | 15 | 2043 | 85.6% | 99.3% | 100.0% | 1 | 3.8% |
| 0.7Q | 15 | 1988 | 11.5% | 40.5% | 71.2% | 15 | 2054 | 62.9% | 90.0% | 90.0% | 0 | 3.3% |
| 0.6Q | 15 | 2028 | 0.0% | 0.7% | 2.6% | 15 | 2068 | 20.2% | 51.3% | 77.1% | 0 | 2.0% |

**Table 14**
Experimental results for calculation 02o100 with different rider loading limits.

| Load | Det | | | | | Rob | | | | | Increase | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| limit | N.R. | Obj | L0 | L1 | L2 | N.R. | Obj | L0 | L1 | L2 | N.R. | Obj |
| 1.0Q | 13 | 1782 | 86.5% | 98.7% | 99.9% | 13 | 1855 | 98.8% | 100.0% | 100.0% | 0 | 4.1% |
| 0.9Q | 13 | 1790 | 48.8% | 82.2% | 97.0% | 13 | 1859 | 83.2% | 98.8% | 100.0% | 0 | 3.9% |
| 0.8Q | 13 | 1794 | 17.2% | 49.0% | 75.3% | 14 | 1873 | 57.4% | 91.4% | 98.7% | 1 | 4.4% |
| 0.7Q | 13 | 1814 | 10.0% | 34.5% | 64.3% | 14 | 1876 | 24.3% | 63.3% | 89.0% | 1 | 3.4% |
| 0.6Q | 13 | 1823 | 0.2% | 0.9% | 4.0% | 14 | 1879 | 15.2% | 44.2% | 73.0% | 1 | 3.1% |




(a) Instance 01o100 (b) Instance 02o100

**Fig. 9.** Service level probabilities for instances 01o100 and 02o100 under different rider load capacity limits.

**Table 15**
Experimental results for instance 01o100 and instance 02o100 with different uncertainty budgets.

| Uncertainty | 01o100 | | | | | 02o100 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| budget | N.R. | Obj | L0 | L1 | L2 | N.R. | Obj | L0 | L1 | L2 |
| 0 | 14 | 1914 | 10.9% | 35.0% | 60.0% | 13 | 1782 | 4.2% | 20.5% | 43.3% |
| 0.2 | 14 | 1968 | 26.3% | 59.7% | 81.5% | 13 | 1778 | 21.6% | 54.5% | 79.3% |
| 0.4 | 15 | 2010 | 49.2% | 87.6% | 97.6% | 13 | 1820 | 41.9% | 80.3% | 95.8% |
| 0.6 | 15 | 2043 | 85.0% | 97.5% | 99.8% | 14 | 1876 | 91.9% | 99.9% | 100.0% |
| 0.8 | 16 | 2046 | 86.6% | 98.4% | 100.0% | 15 | 1891 | 95.3% | 100.0% | 100.0% |
| 1 | 16 | 2050 | 94.5% | 99.9% | 100.0% | 15 | 1896 | 98.9% | 100.0% | 100.0% |

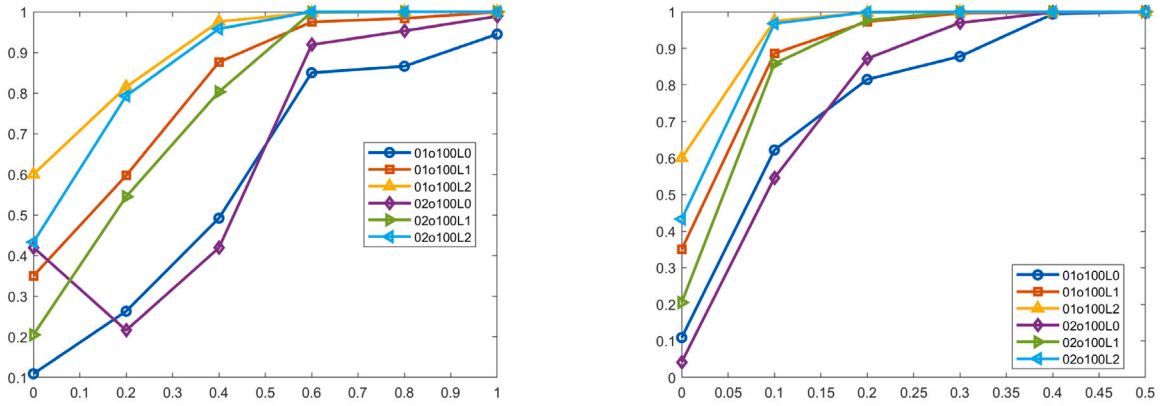
similar situation is observed as the uncertainty budget increases from 0.6 to 0.8. In some cases, the total number of riders remains unchanged, but orders are reassigned within the delivery plan to better handle the increased proportion of uncertainty.

The experimental results also indicate that as the uncertainty budget increases, the cost of the delivery plan rises accordingly. To manage higher levels of uncertainty, it is often necessary to dispatch additional riders or adjust optimal routing, which subsequently leads to an increase in delivery costs. For example, in instance 01o100, with an uncertainty budget of 0.6, the probability of serving all customers is 71.7%, and the probability of serving all but two customers reaches 100%. Although this service level is lower than that achieved with uncertainty budgets of 0.8 and 1.0, it may still be acceptable in certain cases as a cost-saving measure.

Next, we analyze the impact of the uncertainty interval, setting its values to 0, 0.1, 0.2, 0.3, 0.4, and 0.5. An uncertainty interval of 0 represents a deterministic scenario, with the uncertainty budget fixed at 0.5. In this experiment, Monte Carlo simulations were employed under

**Table 16**
Experimental results for instance 01o100 and instance 02o100 with different uncertainty intervals.

| Uncertainty | 01o100 | | | | | 02o100 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| interval | N.R. | Obj | L0 | L1 | L2 | N.R. | Obj | L0 | L1 | L2 |
| 0 | 14 | 1914 | 10.9% | 35.0% | 60.0% | 13 | 1782 | 4.2% | 20.5% | 43.3% |
| 0.1 | 15 | 2024 | 62.2% | 88.6% | 97.5% | 14 | 1826 | 54.6% | 85.8% | 96.8% |
| 0.2 | 15 | 2031 | 81.5% | 97.3% | 99.8% | 14 | 1855 | 87.2% | 97.7% | 99.9% |
| 0.3 | 16 | 2068 | 87.8% | 99.6% | 100.0% | 15 | 1905 | 97.0% | 100.0% | 100.0% |
| 0.4 | 16 | 2097 | 99.4% | 99.8% | 100.0% | 15 | 1940 | 99.8% | 100.0% | 100.0% |
| 0.5 | 16 | 2105 | 100.0% | 100.0% | 100.0% | 16 | 1973 | 99.9% | 100.0% | 100.0% |




(a) Uncertain budgets (b) Uncertain interval

**Fig. 10.** Service level probabilities for instances 01o100 and 02o100 under different uncertainty parameters.

the assumption that order demand follows a normal distribution with a mean equal to the demand r in each instance and a variance of $0.2r$. Similarly, service time was modeled as a variable following the normal distribution with a mean equal to the instance service time $t_s$ and a variance of $0.2t_s$. The experimental results are presented in Table 16.

The results show that even small uncertainty intervals (e.g., 0.1) yield significant service level improvements. In instance 01o100, the probability of serving all customers rises from 10.9% to 62.2%, when the uncertainty interval increases from 0 to 0.1. Similarly, for instance 02o100, the probability of serving all customers rises from approximately 5% to over 50%. These findings highlight the critical role of the uncertainty interval in enhancing service levels.

Additionally, Fig. 10 illustrates the service level trends for both uncertainty budgets and uncertainty intervals. The figure clearly shows that as both factors increase, the service levels consistently improve. However, the uncertainty interval has a more evident effect on service level enhancement compared to the uncertainty budget. When the uncertainty interval reaches 0.2 or higher, the probability of serving all but two customers reaches 100.0% in both instances. At an uncertainty interval of 0.3, the probability of fulfilling all orders reaches 87.8% and 97.0% for instances 01o100 and 02o100, respectively, indicating exceptionally high service levels.

However, this improvement in service level is accompanied by a significant increase in delivery costs. For example, in instance 02o100, increasing the uncertainty interval from 0 to 0.5 results in delivery costs rising from 1782 to 1973, representing a 10.7% increase. In comparison, increasing the uncertainty budget from 0 to 1 leads to a cost increase from 1782 to 1882, leading to a rise of 5.6%. These findings underscore the importance of determining an appropriate uncertainty interval in practical applications to achieve a balance between delivery costs and service levels.

### 5.6.3. Sensitivity analysis of rider labor cost

In the original model design, the objective function only considered the costs associated with e-bike usage and delivery operations, without incorporating the labor cost arising from riders' working hours. This simplification helped reduce the complexity of modeling and solution procedures. However, in real-world food delivery scenarios, rider labor costs are one of the critical factors affecting operational efficiency and cost control. To improve the practical applicability of the model, this study introduces rider labor cost $f_5$, defined as the product of the unit labor cost per time and the rider's actual working time. Accordingly, the total cost of a delivery plan $H$ is expressed as:

$$C'(H) = C(H) + f_5 \sum_{k \in K} (\tau^k_{2n+1} - \tau^k_0) \tag{41}$$

Following the study by Reyes et al. (2018), the rider's hourly wage is initially set to 15. A sensitivity analysis is then conducted using four wage levels (10, 15, 20, and 25) to assess the impact of this parameter on overall scheduling outcomes and the cost structure.

As shown in Table 17, the total delivery cost increases significantly as the unit labor time cost rises from 10 to 25. For instance, in instance 01o100, the total cost increases from 2443 to 3081, representing a 26.1% rise; in instance 02o100, the cost grows from 2312 to 2792, a 20.7% increase. In terms of robustness, the service levels L0, L1, and L2 remain consistently high, indicating that the robustness of the delivery plan is well maintained even with the introduction of labor costs.

Moreover, the number of riders also increases with rising unit time costs. This trend is mainly due to the algorithm's tendency to generate denser, shorter delivery routes when time costs are high, in order to reduce individual rider workload. Although this reduces the flexibility of route scheduling, the algorithm is still able to maintain strong robustness through an effective trade-off between cost control and service level assurance, particularly as L1 and L2 remain nearly 100% in all cases. In instance 01o100, for example, when the unit time cost increases from 0 to 10, the L0 service level rises from 81.5% to 88.8%. This suggests that while the cost increase is modest, robustness actually improves, likely because shorter routes help mitigate the impact of service time variability. However, when the cost rises further to 25, L0 drops to 84.0%, indicating that overly high time costs begin to

**Table 17**
Experimental results for the sensitivity analysis of rider labor cost.

| $f_5$ | 01o100 | | | | | 02o100 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | N.R. | Obj | L0 | L1 | L2 | N.R. | Obj | L0 | L1 | L2 |
| 0 | 15 | 2031 | 81.5% | 97.3% | 99.8% | 13 | 1855 | 87.2% | 97.7% | 99.9% |
| 10 | 15 | 2443 | 88.8% | 98.9% | 99.9% | 14 | 2312 | 83.8% | 98.9% | 100.0% |
| 15 | 15 | 2671 | 87.7% | 99.1% | 100.0% | 14 | 2447 | 81.8% | 98.9% | 100.0% |
| 20 | 16 | 2944 | 84.5% | 98.4% | 100.0% | 15 | 2684 | 84.9% | 98.0% | 99.8% |
| 25 | 16 | 3081 | 84.0% | 98.5% | 99.9% | 15 | 2792 | 83.4% | 97.7% | 99.9% |

constrain route optimization, causing some orders to fall outside their time windows.

From an operational perspective, setting a reasonable unit time cost for riders is crucial. If the cost is set too low, the system may overly focus on route optimization while ignoring rider labor costs. Conversely, if it is set too high, it may reduce the system's overall service capacity and negatively affect user experience. Therefore, when designing dispatch strategies, companies should balance labor costs with customer satisfaction metrics to achieve overall optimization.

## 6. Conclusion

This study addresses the food delivery route optimization problem by employing robust optimization techniques to efficiently generate resilient delivery plans in the presence of stochastic factors. From a modeling perspective, a robust optimization model is developed by incorporating two key stochastic variables—random demand and random service time—based on real-world scenarios. The model aims to minimize platform operating costs under uncertainty by optimizing both order allocation and route planning. To solve this problem, a tailored QL-LNS algorithm is proposed, with its effectiveness validated through experimental results.

The experimental results demonstrate that delivery plans derived from the robust optimization model exhibit superior robustness compared to those from the deterministic model, particularly in handling uncertain demand and random service times. While robust plans may slightly increase platform operating costs and require more riders, they adapt more effectively to stochastic variations. Furthermore, increasing the uncertainty budget and interval enhances the robustness of the solution. However, as the uncertainty range expands, associated costs rise significantly. Therefore, practical applications must carefully select the uncertainty interval to balance robustness and cost effectively.

Future research can be expanded in the following directions. First, the large-scale instances utilized in this study are adapted from Damian's examples for the food delivery problem, each comprising 100 orders (200 nodes). While the proposed algorithm performs effectively at this scale, future work could focus on refining the approach to handle even larger instances, thereby extending its applicability to more extensive platforms. However, this may introduce challenges related to algorithm acceleration techniques, which would require further investigation. Second, this study primarily addresses uncertainties in demand and service time, while other factors, such as traffic conditions and rider availability, are treated as deterministic. Incorporating these additional uncertainties into the model could further enhance its applicability to more complex and dynamic environments. These directions provide valuable opportunities for advancing research in this field.

## CRediT authorship contribution statement

**Liyang Xiao:** Writing – original draft, Methodology, Conceptualization. **Zehao Du:** Writing – original draft, Validation. **Peng Sun:** Writing – review & editing, Conceptualization. **Shi Cheng:** Writing – review & editing, Methodology, Conceptualization.

## Acknowledgments

This research is supported by the National Natural Science Foundation of China (Grant Nos. 72201164, 72101178, 61806119), the "Chen Guang" project funded by the Shanghai Municipal Education Commission (Grant No. 21CGA49), the Natural Science Basic Research Plan in Shaanxi Province of China (Grant No. 2024JC-YBMS-516), and the Fundamental Research Funds for the Central Universities (Grant No. GK202507001).

## Data availability

Data will be made available on request.

## References

Agra, A., Christiansen, M., Figueiredo, R., Hvattum, L. M., Poss, M., & Requejo, C. (2013). The robust vehicle routing problem with time windows. *Computers & Operations Research*, *40*(3), 856–866.

Berbeglia, G., Cordeau, J.-F., Gribkovskaia, I., & Laporte, G. (2007). Static pickup and delivery problems: a classification scheme and survey. *Top*, *15*, 1–31.

Bertsimas, D., & Sim, M. (2003). Robust discrete optimization and network flows. *Mathematical Programming*, *98*(1), 49–71.

Bertsimas, D., & Sim, M. (2004). The price of robustness. *Operations Research*, *52*(1), 35–53.

Brandão, J. (2018). Iterated local search algorithm with ejection chains for the open vehicle routing problem with time windows. *Computers & Industrial Engineering*, *120*, 146–159.

Chen, X., Ulmer, M. W., & Thomas, B. W. (2022). Deep Q-learning for same-day delivery with vehicles and drones. *European Journal of Operational Research*, *298*(3), 939–952.

Chen, P., & Wang, Q. (2024). Learning for multiple purposes: A Q-learning enhanced hybrid metaheuristic for parallel drone scheduling traveling salesman problem. *Computers & Industrial Engineering*, *187*, Article 109851.

Chen, J.-f., Wang, L., Wang, S., Wang, X., & Ren, H. (2022). An effective matching algorithm with adaptive tie-breaking strategy for online food delivery problem. *Complex & Intelligent Systems*, *8*(1), 107–128.

Cheramin, M., Chen, R. L.-Y., Cheng, J., & Pinar, A. (2021). Data-driven robust optimization using scenario-induced uncertainty sets. arXiv preprint arXiv:2107.04977.

Elshaer, R., & Awad, H. (2020). A taxonomic review of metaheuristic algorithms for solving the vehicle routing problem and its variants. *Computers & Industrial Engineering*, *140*, Article 106242.

Endler, K. D., Scarpin, C. T., Steiner, M. T., & Choueiri, A. C. (2023). Systematic review of the latest scientific publications on the vehicle routing problem. *Asia-Pacific Journal of Operational Research*, *40*(06), Article 2250046.

Feng, B., & Wei, L. (2023). An improved multi-directional local search algorithm for vehicle routing problem with time windows and route balance. *Applied Intelligence: The International Journal of Artificial Intelligence, Neural Networks, and Complex Problem-Solving Technologies*, *53*, 11786–11798.

Fowler, J. W., & Mönch, L. (2022). A survey of scheduling with parallel batch (p-batch) processing. *European Journal of Operational Research*, *298*(1), 1–24.

Gabrel, V., Murat, C., & Thiele, A. (2014). Recent advances in robust optimization: An overview. *European Journal of Operational Research*, *235*(3), 471–483.

Ghilas, V., Cordeau, J.-F., Demir, E., & Woensel, T. V. (2018). Branch-and-price for the pickup and delivery problem with time windows and scheduled lines. *Transportation Science*, *52*(5), 1191–1210.

Hamid, M., Nasiri, M. M., & Rabbani, M. (2023). A mixed closed-open multi-depot routing and scheduling problem for homemade meal delivery incorporating drone and crowd-sourced fleet: A self-adaptive hyper-heuristic approach. *Engineering Applications of Artificial Intelligence*, *120*, Article 105876.

Hornstra, R. P., Silva, A., Roodbergen, K. J., & Coelho, L. C. (2020). The vehicle routing problem with simultaneous pickup and delivery and handling costs. *Computers & Operations Research*, *115*, Article 104858.

Hu, C., Lu, J., Liu, X., & Zhang, G. (2018). Robust vehicle routing problem with hard time windows under demand and travel time uncertainty. *Computers & Operations Research*, *94*, 139–153.

Huang, H., Hu, C., Zhu, J., Wu, M., & Malekian, R. (2021). Stochastic task scheduling in UAV-based intelligent on-demand meal delivery system. *IEEE Transactions on Intelligent Transportation Systems*, *23*(8), 13040–13054.

Jahanshahi, H., Bozanta, A., Cevik, M., Kavuk, E. M., Tosun, A., Sonuc, S. B., Kosucu, B., & Başar, A. (2022). A deep reinforcement learning approach for the meal delivery problem. *Knowledge-Based Systems*, *243*, Article 108489.

Jolfaei, A. A., & Alinaghian, M. (2024). Multi-depot vehicle routing problem with roaming delivery locations considering hard time windows: Solved by a hybrid ELS-LNS algorithm. *Expert Systems with Applications*, *255*, Article 124608.

Koç, Ç., Laporte, G., & Tükenmez, İ. (2020). A review of vehicle routing with simultaneous pickup and delivery. *Computers & Operations Research*, *122*, Article 104987.

Lee, C., Lee, K., & Park, S. (2012). Robust vehicle routing problem with deadlines and travel time/demand uncertainty. *Journal of the Operational Research Society*, *63*(9), 1294–1306.

Li, F. C., & Liu, Y. (2009). Generalized expected value model for stochastic programming and its application in vehicle routing problems. In *2009 third international symposium on intelligent information technology application workshops* (pp. 284–287). IEEE.

Li, J., Pardalos, P. M., Sun, H., Pei, J., & Zhang, Y. (2015). Iterated local search embedded adaptive neighborhood selection approach for the multi-depot vehicle routing problem with simultaneous deliveries and pickups. *Expert Systems with Applications*, *42*(7), 3551–3561.

Li, X., Tian, P., & Leung, S. C. (2010). Vehicle routing problems with time windows and stochastic travel and service times: Models and algorithm. *International Journal of Production Economics*, *125*(1), 137–145.

Li, J., Xin, L., Cao, Z., Lim, A., Song, W., & Zhang, J. (2022). Heterogeneous attentions for solving pickup and delivery problem via deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems*, *23*(3), 2306–2315.

Li, C., Zhang, L., Ou, Z., Wang, Q., Zhou, D., & Ma, J. (2022). Robust model of electric vehicle charging station location considering renewable energy and storage equipment. *Energy*, *238*, Article 121713.

Liang, C., Hu, X., Shi, L., Fu, H., & Xu, D. (2022). Joint dispatch of shipment equipment considering underground container logistics. *Computers & Industrial Engineering*, *165*, Article 107874.

Liao, W., Zhang, L., & Wei, Z. (2020). Multi-objective green meal delivery routing problem based on a two-stage solution strategy. *Journal of Cleaner Production*, *258*, Article 120627.

Liberatore, F., Righini, G., & Salani, M. (2011). A column generation algorithm for the vehicle routing problem with soft time windows. *4or*, *9*, 49–82.

Lin, B., Ghaddar, B., & Nathwani, J. (2021). Deep reinforcement learning for the electric vehicle routing problem with time windows. *IEEE Transactions on Intelligent Transportation Systems*, *23*(8), 11528–11538.

Liu, Y. (2019). An optimization-driven dynamic vehicle routing algorithm for on-demand meal delivery using drones. *Computers & Operations Research*, *111*, 1–20.

Liu, X., Chen, Y.-L., Por, L. Y., & Ku, C. S. (2023). A systematic literature review of vehicle routing problems with time windows. *Sustainability*, *15*(15), 12004.

Liu, H., Hao, P., Liao, Y., Tanvir, S., Boriboonsomsin, K., & Barth, M. J. (2024). Eco-friendly crowdsourced meal delivery: A dynamic on-demand meal delivery system with a mixed fleet of electric and gasoline vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 11397–11410.

Liu, R., & Jiang, S. (2022). A variable neighborhood search algorithm with constraint relaxation for the two-echelon vehicle routing problem with simultaneous delivery and pickup demands. *Soft Computing*, *26*(17), 8879–8896.

Liu, R., Tao, Y., & Xie, X. (2019). An adaptive large neighborhood search heuristic for the vehicle routing problem with time windows and synchronized visits. *Computers & Operations Research*, *101*, 250–262.

Liu, R., Xie, X., Augusto, V., & Rodriguez, C. (2013). Heuristic algorithms for a vehicle routing problem with simultaneous delivery and pickup and time windows in home health care. *European Journal of Operational Research*, *230*(3), 475–486.

Lu, H., Hu, Y., Wang, S., Zhou, P., & Cheng, S. (2025). DTAS: An adaptive critical scenarios generation method for decision boundary assessment. *IEEE Transactions on Cognitive and Developmental Systems*, 1–14.

Lu, H., Wang, S., Cheng, S., & Shi, Y. (2024). Surrogate-based decision boundary evolutionary sampling method for autonomous systems. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 1–15.

Lyu, Z., & Yu, A. J. (2023). The pickup and delivery problem with transshipments: Critical review of two existing models and a new formulation. *European Journal of Operational Research*, *305*(1), 260–270.

Ning, C., & You, F. (2019). Optimization under uncertainty in the era of big data and deep learning: When machine learning meets mathematical programming. *Computers & Chemical Engineering*, *125*, 434–448.

Powell, W. B. (2019). A unified framework for stochastic optimization. *European Journal of Operational Research*, *275*(3), 795–821.

Qin, W., Zhuang, Z., Huang, Z., & Huang, H. (2021). A novel reinforcement learning-based hyper-heuristic for heterogeneous vehicle routing problem. *Computers & Industrial Engineering*, *156*, Article 107252.

Reyes, D., Erera, A., Savelsbergh, M., Sahasrabudhe, S., & O'Neil, R. (2018). The meal delivery routing problem. *Optimization Online*, *6571*, 2018.

Salehi Sarbijan, M., & Behnamian, J. (2023). Emerging research fields in vehicle routing problem: a short review. *Archives of Computational Methods in Engineering*, *30*(4), 2473–2491.

Souyris, S., Cortés, C. E., Ordóñez, F., & Weintraub, A. (2013). A robust optimization approach to dispatching technicians under stochastic service times. *Optimization Letters*, *7*, 1549–1568.

Taş, D., Jabali, O., & Van Woensel, T. (2014). A vehicle routing problem with flexible time windows. *Computers & Operations Research*, *52*, 39–54.

Ulmer, M. W., Thomas, B. W., Campbell, A. M., & Woyak, N. (2021). The restaurant meal delivery problem: Dynamic pickup and delivery with deadlines and random ready times. *Transportation Science*, *55*(1), 75–100.

Wang, C., Mu, D., Zhao, F., & Sutherland, J. W. (2015). A parallel simulated annealing method for the vehicle routing problem with simultaneous pickup–delivery and time windows. *Computers & Industrial Engineering*, *83*, 111–122.

Wang, A., Subramanyam, A., & Gounaris, C. E. (2022). Robust vehicle routing under uncertainty via branch-price-and-cut. *Optimization and Engineering*, *23*, 1895–1948.

Wang, S., Sun, W., & Huang, M. (2024). An adaptive large neighborhood search for the multi-depot dynamic vehicle routing problem with time windows. *Computers & Industrial Engineering*, *191*, Article 110122.

Wu, H., Gao, Y., Wang, W., & Zhang, Z. (2023). A hybrid ant colony algorithm based on multiple strategies for the vehicle routing problem with time windows. *Complex & Intelligent Systems*, *9*, 2491–2508.

Wu, L., Hifi, M., & Bederina, H. (2017). A new robust criterion for the vehicle routing problem with uncertain travel time. *Computers & Industrial Engineering*, *112*, 607–615.

Xue, G., Wang, Z., & Sheu, J.-B. (2025). Meal pickup and delivery problem with appointment time and uncertainty in order cancellation. *Transportation Research Part E: Logistics and Transportation Review*, *193*, Article 103845.

Xue, G., Wang, Z., & Wang, Y. (2024). The restaurant delivery problem with uncertain cooking time and travel time. *Computers & Industrial Engineering*, *190*, Article 110039.

Yang, J., & Li, Y. (2025). A multicommodity pickup and delivery problem with time windows and handling time in the omni-channel last-mile delivery. *International Transactions in Operational Research*, *32*(3), 1524–1565.

Yildiz, B., & Savelsbergh, M. (2019). Provably high-quality solutions for the meal delivery routing problem. *Transportation Science*, *53*(5), 1372–1388.

Zhang, T., Chaovalitwongse, W. A., & Zhang, Y. (2012). Scatter search for the stochastic travel-time vehicle routing problem with simultaneous pick-ups and deliveries. *Computers & Operations Research*, *39*(10), 2277–2290.

Zhang, T., Cheng, J., & Zou, Y. (2024). Multimodal transportation routing optimization based on multi-objective Q-learning under time uncertainty. *Complex & Intelligent Systems*, *10*, 3133–3152.

Zhen, L., Baldacci, R., Tan, Z., Wang, S., & Lyu, J. (2022). Scheduling heterogeneous delivery tasks on a mixed logistics platform. *European Journal of Operational Research*, *298*(2), 680–698.

Zhen, L., Gao, J., Tan, Z., Laporte, G., & Baldacci, R. (2023). Territorial design for customers with demand frequency. *European Journal of Operational Research*, *309*(1), 82–101.

Zhen, L., Wu, J., Wang, S., He, X., & Tian, X. (2025). Courier routing for a new last-mile logistics service. *IISE Transactions*, *57*(8), 957–975.

Zheng, J., Wang, L., Wang, S., Chen, J.-f., Wang, X., Duan, H., Liang, Y., & Ding, X. (2022). Modeling stochastic service time for complex on-demand food delivery. *Complex & Intelligent Systems*, *8*, 4939–4953.

Zheng, J., Wang, L., Wang, L., Wang, S., Chen, J.-F., & Wang, X. (2022). Solving stochastic online food delivery problem via iterated greedy algorithm with decomposition-based strategy. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, *53*(2), 957–969.